Neural Networks That Don't Lose Track: Exact Digital State Without Chain of Thought

Neo [surname][1*]

[1*]Affiliation TBD.

Corresponding author(s). E-mail(s): TBD;

Abstract

Neural networks that track a state lose track of it beyond their training length. We prove that a loss evaluated only where the training data go cannot see how errors evolve elsewhere, and that next-token likelihood prices a slow drift at a cost carrying the model's own error rate, so robustness must come from the architecture or an off-data probe. A closure onto a fixed sphere after every event keeps small recurrent trackers exact in several settings with a readout loss alone. Training the state to meet a history certified equivalent by a checker is needed on counting. On words in the permutation group $\boldsymbol{S_5}$, perturbing the state along its fastest-growing direction raises noise tolerance and beats isotropic perturbation at equal dose, but its gain over none does not survive multiple-test correction. With all three, trackers keep a digital state, settling onto a finite set of exact codes, without a chain of thought: they are never wrong on held-out problems, on all 10 seeds, at 64 to 256 times their training length, in five settings, and snapping the Lean trackers to the codes gives automata verified exact on every input. Words repeating one short pattern still break most $\boldsymbol{S_5}$ seeds.

Keywords: state tracking, length generalization, error correction, Myhill–Nerode theorem, recurrent networks, language models

1 Introduction

On our small test sets (1 to 5 problems per model and cell), three frontier reasoning models trace programs of 1,024 statements and Lean rewrite chains of 1,024 steps without error. They fail on words of 1,024 moves in the symmetric group S_5 (4 of 15 exact at a 64,000-token budget and 3 of 9 at their largest; 0 of the 17 misses are budget stops), on additions of 4,096 digits (4 of 9; 3 of the 5 misses are budget stops) and on

a Rubik's cube turned 1,024 times (0 of 8; 2 answered inside the budget). Their per-step errors compound with length [1–3], and a small tracker trained by next-token prediction on S_5 words of 16 moves falls to chance at length (Fig. 1a).

We prove that no loss evaluated only on the states the training data reach can tell apart smooth trackers whose errors evolve differently away from them, and that next-token likelihood prices a rare per-step state error at a cost carrying the model's own error rate as a factor (Theorem 1). The theorem does not say what gradient descent finds, only that the loss gives no reason to find contraction. Robustness at length must come from the architecture, through a closure that maps the state back to a fixed set after every event, or from a probe that evaluates the transition off the reached states.

A closure onto a centred sphere, trained with a readout loss alone, is exact on every seed in addition (10), Lean rewrite chains (10) and S_5 words read through a frozen language model (10), and on 9 of 10 in reversible programs, but not on S_5 words from scratch or on counting (4 of 10; Table 1). There we add Nerode training, motivated by a robust form of the Myhill–Nerode theorem (Theorem 2): a checker the domain already has, an interpreter, a proof kernel or a simulator, certifies that two histories end in the same state, and we train the model to agree after perturbing its state along the direction in which errors grow fastest (Fig. 1c). Certified equivalence is needed on counting and stories. On S_5, aiming the perturbation raises noise tolerance and beats an isotropic perturbation of equal dose, but its gain over none does not survive Holm correction. With the closure and Nerode training, trackers are never wrong on any held-out problem, on every seed, at 64 to 256 times their training length in five settings; S_5 words that repeat one short pattern still break most seeds. We call their state digital: it settles onto a finite set of exact codes, one per state of the task, and snapping to the nearest code extracts an automaton, verified exact on Lean. This describes the state, kept without a chain of thought, not general digital computation.

Length generalisation on formal tasks follows the Chomsky hierarchy [4]; shallow transformers learn shortcuts to automata that are brittle out of distribution [5]; state-space models share the transformer's limits on permutation tracking [6]; and negative eigenvalues and products of Householder reflections let linear recurrences track parity and more [7, 8]. Training noise, online clustering and attractor denoising settle small recurrent networks into discrete states [9–12]; error control rather than expressivity limits state tracking [13]; attractor dynamics generalise addition far past the training length [14]; and a Lyapunov tube loss bounds hidden-state deviations [15]. Under bounded analog noise, recurrent networks recognise only regular languages [16], and robust finite-state computation forces the minimal-automaton organisation [17].

2 Likelihood is blind

When the head's rows have equal norm and no bias, next-token likelihood is the readout loss at low temperature on the closed view (Methods). No objective on reached states constrains the Jacobians along a word, and the readout's restoring force carries its own error probability, which falls with the loss (Theorem 1; Fig. 2a). If the event maps do not expand distances, a drift within ρ of the exact loss can fail at every $N > mL/\log(1+\rho/\bar{s})$, and one with the same per-event miss whose margin falls γ

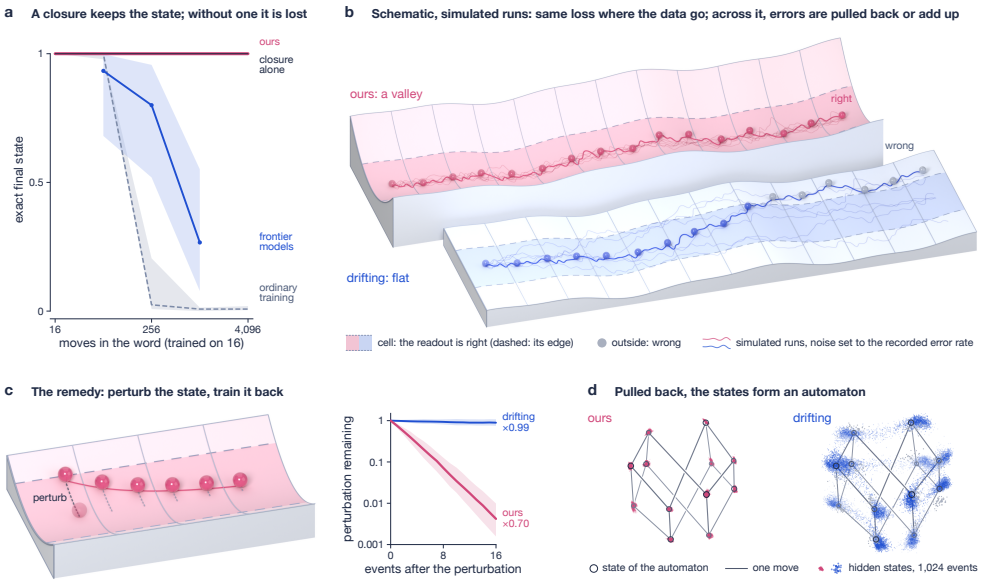


Fig. 1 A loss that fits the training data is blind across the path; a perturbation makes it see. a, S_5 words read as text, after training on 16 moves. Frontier reasoning models (pooled at a 64,000-token budget, 5 problems per model and length; band: exact Clopper–Pearson 95% interval) and ordinary training of the frozen-model tracker (dashed: median of 10 seeds, none exact; band: their range) lose the state and fall towards chance. M, on the frozen Qwen2.5-Coder-0.5B with a trained state slot, is exact at every length to 4,096 on 10 of 10 seeds. The closure with a readout loss alone is drawn along M's curve and is exact at every length on 10 of 10 seeds, so this panel contrasts a closure with none. **b,** Theorem 1 as a picture (schematic; the runs are simulated). Where the training data go, down the middle of each readout cell (tint; dashed: the edge where the readout turns wrong), the two landscapes have the same heights and so the same loss. Across it, M's is a valley that pulls every error back (measured factor 0.70 per event); the drifting tracker's is flat (0.99), so the same errors add up until the state leaves its cell. Runs use the measured factors and noise set to the drifting tracker's recorded error rate. **c,** The remedy, on a close-up of the valley in b: perturb the state off the middle of its cell and require it to return. Right, measured in the from-scratch S_5 trackers trained with M: the perturbation remaining after each event, as a fraction of its size (median; band: quartiles). **d,** Hidden states of the trackers in c on words over $(1\,2)$, $(2\,3)$ and $(4\,5)$, which generate $S_3 \times C_2$. M's tracker is aligned onto the drifting tracker by a rotation, a reflection and one scale, and both are drawn on one skeleton, the mean of their aligned class centres (rings: the automaton's states; lines: one move; 3D projection). M's states condense onto the automaton and are exact on 512 of 512 words of 1,024 moves; the drifting tracker's states are smeared around it (grey: read wrong; 481 of 512 exact).

times more slowly beyond γ times that length. The argument covers every objective reading the transition at finitely many points to finite order (Corollary 1), certified equivalence and our perturbation term included: none certifies exactness when an input passes through a view outside those points.

A weight-space interpolation on Lean illustrates this without proving it (Fig. 2b). Cross-entropy does not prefer the exact network on formal languages [18], and next-token success does not imply a coherent world model [19].

3 A sufficient condition, and Nerode training

Three conditions on a closed tracker suffice for exactness at every length (Theorem 2, Methods): balls inside the right readout cells; blocks of k events that carry each ball into a smaller copy of the next while the states in between stay in their readout cells; and every history of one short length inside its class's ball. With the radius sent to zero and the block to one event, the clauses say that the task's minimal automaton is realised inside the tracker [20, 21]. Length-independent tracking at finite precision is equivalent to each event carrying every cell of a partition into the next with a margin [22]; the block form adds a block length, with a rate λ in place of the margin.

In a trained S_5 tracker the conditions hold on samples, not uniformly (Fig. 2c–f): only a fraction 0.799 of one-event images of boundary points lie inside the next ball, yet every sampled image inside a block is decoded correctly, a block of 4 events carries a fraction 0.998 into the next ball (0.81 in a drifting tracker), and every final state at every tested length lies inside its class's ball.

Nerode training has three parts (Extended Data Fig. 5). The readout cross-entropy enforces the cell clause. The centred closure, part of the architecture, makes the next view a function of the current view and the event. The third part perturbs that view along its forward direction and asks it to meet a history the checker certifies as equivalent,

$$\mathcal{L} = \mathrm{CE} + \mu\big(\tfrac{1}{2}|\bar{z} - z(v)|^2 + G\,\tfrac{1}{2}|D|^2\big), \qquad \bar{z} \pm D = f_{x_e}\big(\kappa(z_{e-1}, \pm\sigma\psi_{e-1})\big). \tag{1}$$

Here f is the event map, z_{e-1} the view before the last event x_e, ψ its forward direction, κ the closure, σ the perturbation size (a chord between views), v a certified equivalent history and G a gain. At $G = 1$ the bracket is the mean centred cosine distance of the two perturbed images to $z(v)$ (Lemma 1). The first term merges the perturbed pair with the equivalent history; the second is the secant growth of one event along ψ. It reads one secant per history and does not bound the growth at every event: trained F trackers still expand some direction in one event (median top singular value 1.73) while contracting along runs. On S_5 held-out problems a window of several events acted as a gain on the secant term, not as a horizon (Methods). Isotropic penalties act on the whole Jacobian [23, 24]; we aim ours by the power iteration of virtual adversarial training [25] and keep the top factor below one only inside the balls, because a globally contracting network, as in contraction-constrained recurrent designs [26–28], forgets its state [29]. We call (1) F, and M its windowed predecessor over 4 events (Methods). Table 1 uses F; the mechanism measurements in Figs. 1–4 use M or our earlier recipe, as each caption states.

4 Results

4.1 When the closure is enough

Trackers are recurrent modules of width 128 that read each event as tokens or as text through a frozen Qwen2.5-Coder-0.5B or Qwen2.5-0.5B model [30, 31] (Methods). A run is exact at a length if its final state is right on at least 0.99 of 512 held-out

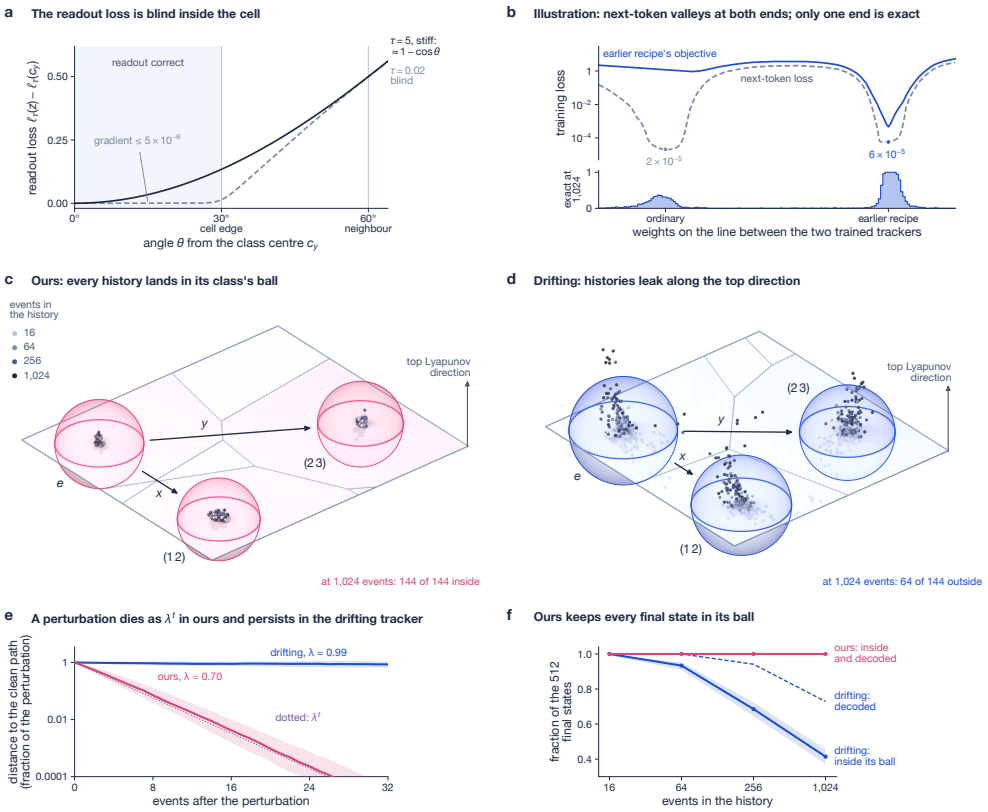


Fig. 2 The theorems, seen in trained trackers. a, The readout loss is blind inside its cell at low temperature, where its gradient carries the error probability, and stiff at high temperature, where it tends to $1 - \cos\theta$ (computed exactly on a centred code). **b,** An illustration, not evidence: on the straight line in weight space between an ordinarily trained Lean tracker and one trained with our earlier recipe, the next-token loss has a valley at each end, as interpolations between trained networks generally do; exactness after 1,024 rewrites separates them, and so does the earlier recipe's objective. **c, d,** The balls of S_5 trackers in 3D: M (top factor 0.70) keeps every drawn history inside its class's ball at 1,024 events; a drifting tracker trained with next-state prediction (0.99) leaks histories along its top Lyapunov direction. Points are projections; counts use true distances. **e,** A perturbation dies as λ^t under M and persists in the drifting tracker (median and quartiles; violet: λ^t). **f,** M keeps every final state of 512 inside its ball at every length; the drifting tracker loses them and decodes fewer (Wilson bands).

problems drawn from the training distribution; a problem is never wrong if the state is right after every event.

The closure with a readout loss alone is exact on every seed in addition (10 of 10, 95% interval 0.69–1.00), Lean (10 of 10, 0.69–1.00; 9 at 4,096 rewrites) and S_5 read through the frozen model (10 of 10), and on 9 of 10 in reversible programs (0.55–1.00); without the closure, the readout loss is exact nowhere except on 2 of 3 S_5 seeds from a separate study (Table 1). The closure falls short on S_5 from scratch (3 of 10) and counting (4 of 10, 0.12–0.74), where certified equivalence is needed. Our earlier recipe falls short only on S_5 from scratch (15 of 20). With F, trackers are exact on every

Table 1 Exact at length, by setting and objective. Seeds exact at the test length (final state right on at least 0.99 of 512 problems) over seeds run; – marks a cell with no record. F, Nerode training in its one-event form; M, its windowed form (Lean closes by the RMS norm); NS+EQ, our earlier recipe (readout, next-state prediction, certified equivalence and centred closure); O+Nc, the readout loss with the centred closure; O, the readout loss alone. The S_5 O entry comes from a separate study with its own protocol. λ, median top Lyapunov factor per event under F. Train, the longest training history; test, the evaluation length, in events. At 10 of 10 seeds the exact Clopper–Pearson 95% interval is 0.69–1.00.

Setting	Train	Test	Ratio	F	M	NS+EQ	O+Nc	O	λ
Reversible programs (A_{16})	16	4,096	256×	10/10	10/10	10/10	9/10	0/10	0.06
S_5 words, from scratch	16	1,024	64×	10/10	20/20	15/20	3/10	2/3	0.59
Lean rewrite chains	8	1,024	128×	10/10	10/10	10/10	10/10	0/10	0.08
Long addition	32	4,096	128×	10/10	10/10	10/10	10/10	0/10	0.14
Counting	32	4,096	128×	10/10	10/10	10/10	4/10	0/10	0.18
S_5 read by the frozen model	16	4,096	256×	–	10/10	–	10/10	0/10	–
Rubik's cube	32	4,096	128×	–	10/10	3/3	2/3	0/3	–

seed in five settings at 64 to 256 times the training length (Table 1; Lean also at 4,096 rewrites, addition also when carries run the whole length), and on these seeds no test problem is ever wrong.

That is not every input. On S_5 words that repeat one pattern of up to four moves for 1,024 moves, F, in a separate registered run, fails at least one of 130 such words on 8 of 10 seeds (96 of 1,300 seed–word pairs), and M fails a repeated single move on 13 of 20 seeds (Supplementary Information). Stories, where an object's final place is set by its last move, and eight-variable programs, whose final state a short suffix often determines, test less than their length suggests. F as above is exact on 5 of 10 story seeds and 7 of 10 program seeds; retunes chosen after the test sets were scored reach 9 of 10 and 9 of 10, and 8 and 9 on fresh test sets, the programs after retraining (Supplementary Information).

4.2 Which part is needed where

We drop one part of M at a time and retrain 10 seeds per setting (Fig. 3). Of the 22 one-sided Fisher tests, 7 survive Holm correction at 0.05: dropping the closure in reversible programs, addition, counting, stories and programs, and dropping certified equivalence in counting and stories. Without the closure, S_5 keeps 9 of 10 seeds. Dropping aimed contraction lowers S_5 to 5 of 10 (Holm-adjusted $p = 0.211$) and changes nothing elsewhere, and M against our earlier recipe on S_5 gives a Holm-adjusted $p = 0.283$. At equal dose, aiming wins, as registered: an isotropic perturbation with M's window, size and weight is exact on 14 of 20 S_5 seeds against M's 20 of 20, one in F's form on 3 of 10 against F's 10 of 10, and both fail far more repeated words (Supplementary Information). Lean separates no part.



Fig. 3 Which part is needed where. a, Drop one part of M, retrain and test at the setting's test length: one disc per seed, shaded where fewer seeds are exact than with all parts at one-sided Fisher $p < 0.05$ before correction; 7 of the 22 tests survive Holm correction (Supplementary Information). The S_5 and Lean trackers are trained from scratch. **b,** Each failure takes one of the theorem's two routes. Without equivalence, equivalent histories land apart, at pair distances many times those of M; the runs that fail without aimed contraction or the closure sit at a top Lyapunov factor near one.

4.3 Reversible programs and the word problem of A_{16}

A reversible program applies a NOT, CNOT or Toffoli gate to four bits per statement; these gates generate the alternating group A_{16} on the sixteen states (Methods), so tracking a program is the word problem of a non-solvable group, complete for NC1 [32] and beyond constant-depth transformers without chain of thought unless TC0 equals NC1 [33, 34]. F is never wrong on any test program at 4,096 statements on all 10 seeds, with a top factor of 0.06. At 4,096 statements the frontier models solve 2 of 6 programs at the standard budget (4 budget stops) and 7 of 9 at their largest, using up to 103 thousand output tokens per program; our trackers generate none (Extended Data Fig. 7).

4.4 Through a frozen language model

Read as text, S_5 words are tracked exactly at 4,096 moves (256×; Fig. 1a) by M on 10 of 10 seeds. A cube tracker with one slot per piece and shared weights, which builds the product structure in, is never wrong at 4,096 and at 10,000 moves on all 10 seeds (Extended Data Fig. 8).

The language model's own codes can carry a single skill. A linear map on the frozen model's layer-12 codes, trained teacher-forced with a readout loss and no Nerode term, runs free and is never wrong on 512 of 512 problems at every tested length to 4,096 events, in all 10 runs on cards (S_5 as swaps of named cards) and reversible

programs. This shows that the fixed codes are linearly related across events, not that the language model tracks state. One such step per task is never wrong on all 21 seeds of seven tasks and on all cube and Lean seeds, but not on eight-variable programs (Supplementary Information). Adding F ($\sigma = 0.316$) broke the free run on five of five card seeds and four of five program seeds, whose median cell radii along ψ are 0.18 and 0.40; no smaller size was tried.

4.5 Errors die at a measured rate

F's top Lyapunov factor per event runs from 0.06 to 0.59 across settings (Table 1). A perturbation of a trained S_5 tracker dies as λ^t with $\lambda = 0.70$ and persists in a drifting one ($\lambda = 0.99$; Fig. 2e), though contraction alone does not predict exactness (7 of 189 exact runs sit at $\lambda \geq 0.97$).

Contraction buys noise tolerance. With Gaussian noise of relative size ε added after every event, M stays exact at 1,024 events up to a median ε_{99} of 0.143 on all 20 S_5 seeds, and its worst seed is at or above every other arm's median, as registered. Over their seeds exact without noise, our earlier recipe reaches 0.080 (Fig. 4a) and M without aimed contraction (EQ+Nc) 0.075. On Lean, M reaches 0.170 against 0.102 ($n = 3$ for the earlier recipe). Theorem 2's bound covers perturbations at block boundaries only; it is vacuous on S_5 and orders the Lean arms the wrong way round, so the tolerance is empirical.

Replacing the state after every event by the nearest class code extracts a finite automaton that can be checked exhaustively [35–37]; the check verifies that automaton, not the continuous tracker. On Lean all 10 of 10 extracted automata are verified exact on every input at every length (Fig. 4b). On S_5 none was: training with a snap inside the loop certified none of 20 trackers, contract-then-snap 0 of 10, and a longer run 0 of 10. On its exact seeds contract-then-snap made 0 readout errors but left 351 to 4,532 edges unsnapped, and the longer run's searches all hit their budgets. A certificate for the continuous trackers is open.

5 Discussion

A closure onto a fixed sphere often suffices; certified equivalence is needed on counting and stories; and on S_5 from scratch, the task it was developed on, aimed contraction raises noise tolerance and beats an isotropic perturbation of equal dose, while its gain over none does not survive Holm correction. In our settings the labels name the full state, so we do not show that a checker supplies information that labels cannot.

Our settings are synthetic and finite-state, and seeds are few (10 per setting, 3 for some baselines). We trained no tuned linear recurrence [7, 8] on S_5. Repeated words break most S_5 seeds; a counterexample loop over such words cut the failing pairs of a window of 4 events from 155 to 59 but not F's (105 against 96), so on these inputs a window may be more than a gain. On the frozen model's own codes F broke most runs. The extracted automata are verified only on Lean, and no continuous tracker is certified. We add aimed contraction to a line from finite automata in neural nets to recurrent nets on regular languages [38–42].

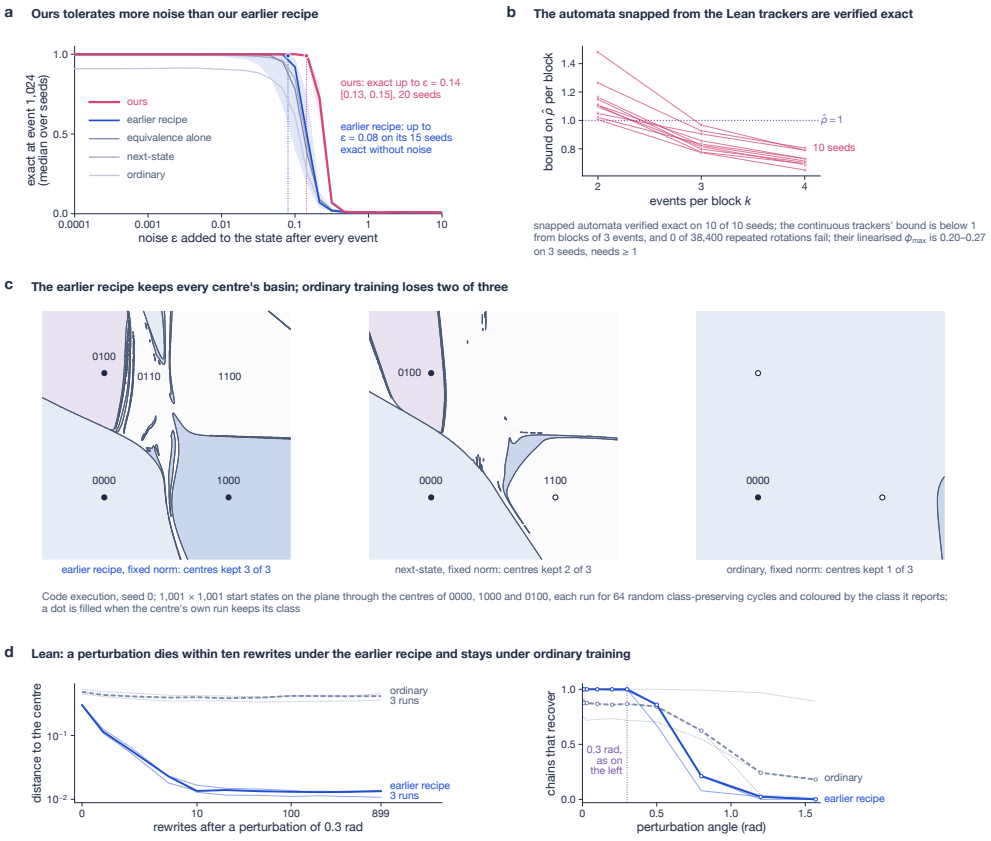


Fig. 4 Error correction. a, M tolerates more noise than our earlier recipe: Gaussian noise after every event, exactness at 1,024 events against relative noise size; median ε_{99} 0.143 [0.128–0.149] on 20 of 20 S_5 seeds, against 0.080 over the earlier recipe's 15 seeds exact without noise (0.071 with $\varepsilon_{99} = 0$ imputed for the other 5), a ratio of 1.8. **b,** The automata extracted from the Lean trackers by snapping are verified exact on every input at every length (10 of 10); the bound on the joint spectral radius of the continuous trackers falls below one from blocks of three events. **c,** Basins on the plane through three class centres of a program tracker: the earlier recipe with the fixed norm keeps every centre's basin; ordinary training does not. **d,** Lean: a perturbation dies under the earlier recipe and stays under ordinary training.

6 Methods

Statements

The code's closure keeps the centred state at norm $\sqrt{d}$, with d its width. We write $z(u)$ for the view of a history u, that state divided by its norm, on the centred unit sphere; radii, σ, ε and the distances in (1) are chords between views. We write $c_1, \ldots, c_C$ for class codes on that sphere with $\sum_j c_j = 0$, and

$$\ell_\tau(z, y) = \tau\big(\mathrm{CE}(\langle z, c_\cdot \rangle/\tau,\ y) - \log C\big)$$

for the readout loss at temperature τ, with error probability $s = 1 - p_y(z)$. A closed tracker acts by maps F_x, one per event, F_w the map of a word w. The task's minimal automaton has states q, outputs $o(q)$ and transitions $q \cdot w$, with a code c_q, a radius $r_q > 0$, a block length $k \geq 1$ and a rate $\lambda \in [0, 1)$.

Theorem 1 (Likelihood is blind) (a) *Let a closed tracker's views on the training data take finitely many values. For any linear maps between the tangent spaces at those views there is a smooth tracker, not necessarily one the network family can represent, that agrees with it at every reached view and has those maps as its Jacobians there. Every objective that reads the transition only through its values at reached views takes the same value at both trackers.* (b) *At every $\tau > 0$, $\|\nabla_z \ell_\tau\| \leq 2s$ and $\nabla_z^2 \ell_\tau \preceq (4s/\tau) I$. As $\tau \to \infty$, $\ell_\tau(z, y) \to -\langle z, c_y \rangle$ uniformly.* (c) *Suppose every learned event map is non-expansive on a set that contains the exact and the learned views, and misses the exact next view by at most ε. Let K be the Lipschitz constant of the logit vector in the view, m the exact tracker's smallest logit margin and $\bar{s}$ its mean error probability over the loss's positions. Then at training length L the cross-entropy exceeds the exact tracker's by at most $\bar{s}\,(e^{\sqrt{2}K\varepsilon L} - 1)$, and the readout is exact to length N when $\sqrt{2}K\varepsilon N < m$. The bound is sharp: some non-expansive transitions with this ε fail at every $N > m/(\sqrt{2}K\varepsilon)$.*

Corollary 1 *Let an objective read values, and finitely many derivatives, of the transition at finitely many points, possibly chosen from what it has read. For any closed tracker with at least two readout cells, any radii r_q and any point of a ball $B(c_q, r_q)$ that the objective does not read, there is a smooth closed tracker with the same objective value whose event map sends that point out of the readout cell of its next state. Reached-state objectives and F are such objectives, so none of them certifies (C_r) for given radii. Likewise, if an input reaches a view that the objective does not read and an event can follow that input, some smooth closed tracker with the same objective value is not exact.*

Theorem 2 (Robust Myhill–Nerode) *Suppose that (O_r) every ball $B(c_q, r_q)$ lies in the readout cell of $o(q)$; (C_r) for every state q and applicable word w, $F_w(B(c_q, r_q))$ lies in the readout cell of $o(q \cdot w)$ if $|w| < k$, and in $B(c_{q \cdot w}, \lambda r_{q \cdot w})$ if $|w| = k$; (E_r) for some prefix length p, the readout is right on every word shorter than p, and the view reached by every word of length p lies in the ball of the state it reaches. Then the readout is exact on every input at every length. It stays exact along any run that is unperturbed through the prefix, follows the tracker inside blocks, and is moved by at most $(1 - \lambda) \min_q r_q$ at each block boundary after the prefix, the first being $p + k$. The converse does not hold in general.*

Lemma 1 *With $\bar{z}$ and D as in (1), $\frac{1}{2} \sum_{s=\pm 1} d\big(\bar{z} + sD,\, z(v)\big) = \frac{1}{2}|\bar{z} - z(v)|^2 + \frac{1}{2}|D|^2$, where $d(a, b) = \frac{1}{2}|a - b|^2$ is the centred cosine distance.*

Settings and certified pairs

Reversible programs. Each statement applies a NOT, CNOT or Toffoli gate to four bits; the state is the bit string. Training programs have 4 to 16 statements; test sets have 512 programs per length up to 4,096. A partner is an independent program that the interpreter runs to the same final state. `checks/a16.py` computes

the order of the group the gates generate and confirms that it is $16!/2$. S_5 **words.** Each event is a transposition, rendered among distractor tokens; the state is the product. From scratch, the tracker reads tokens (218,488 parameters, 4,000 steps); through the language model, it reads each move as a line of text. Partners are independent words with the same product. **Lean rewrite chains.** Each event is a rewrite tactic on an arithmetic expression in natural-number variables, and the state is the expression; equivalence of two chains is certified by the Lean 4 kernel [43]. Partners pair each prefix with a direct encoding of the expression it reaches, followed by the chain's remaining events. Lean trackers have 227,773 parameters and train for 12,000 steps. **Addition and counting.** Long addition reads aligned digit pairs, least significant first; the state is the pair of the current sum digit and the carry, and the readout reads both. Counting reads a marked token among distractors and outputs the count modulo ten, the finite-state part of counting. Partners come from Python integers, with every length from one to the training length. Training runs 3,000 steps on 4 to 32 items. **Rubik's cube.** Each event is one of the eighteen face turns, read as text through the frozen Qwen2.5-0.5B. The tracker has one slot per movable piece with shared weights and outputs each piece's position and orientation; a simulator certifies partners, different words that reach the same cube state (4,000 steps, batch 32). **Stories and programs.** Stories move objects between containers in varied English; programs assign eight variables. Both are described, with their results, in the Supplementary Information.

Trackers

A tracker holds a state of width 128, starting from a learned vector, that a transition network updates once per event, followed by the setting's closure: the state cross-attends to the event's encodings, which pools the tokens of one event, and a residual MLP follows (Lean applies this block four times per event). Encodings come from token embeddings (S_5, Lean) or from the frozen language model's layer-12 hidden states (width 896) of each event's text. The readout is a layer norm followed by a linear head. Trackers on the frozen code model have 578,448 trainable parameters. The Supplementary Information gives the equations, the losses with their stop-gradients and the construction of partners in each setting.

Objectives

$\mathbf{F}$ is equation (1) with $\sigma^2 = 0.1$ ($\sigma = 0.32$), $\mu = 1$ and $G = 4$, added to the readout cross-entropy averaged over every event. Whether a window of events adds more than weight on held-out problems was a registered question on S_5: one event at $G = 4$ is exact on 10 of 10 seeds, a window of 4 events with the weight scaled down by the same factor on 7 of 10, and one event at $G = 1$ on 8 of 10. The direction ψ is the forward direction at the state before the last event, found by power iteration of the closed tangent map (forward-mode derivatives) along the word's whole past, with stop-gradients on parameters and states. The perturbed view $\kappa(z, s\sigma\psi)$ is the view moved by a chord of about σ along $\pm\psi$ and closed again. The partner state $z(v)$ is that of an independent certified-equivalent history; it, the base state and the perturbed images carry gradients. $\mathbf{M}$ is $\mathcal{L}_{\text{out}} + \mu\,\text{EQ} + 10 \cdot \text{AIM}$. EQ is the centred

cosine distance between the final states of a history and its certified partner. AIM moves a reached state by a chord $\sigma = 0.1$ along the forward direction (power iteration with a burn-in of 8 events), runs it through the next 4 events with live parameters and penalises its distance to the clean state, with stop-gradients on the direction, the base state and the target. Lean uses 3 events, a burn-in of 3 and the RMS closure $h \leftarrow \sqrt{d}\, h/\|h\|$, registered before the centred closure and not rerun. **Baselines.** O is the readout loss alone; O+N and O+Nc add the RMS or centred closure; EQ+Nc adds certified equivalence; NS+N is next-state prediction with the RMS closure; NS+EQ (FULL in the records) is our earlier recipe: readout, next-state prediction, certified equivalence and closure. **Optimisation.** AdamW [44] with warm-up and cosine decay to a learning rate of 0.001, batch 64 unless stated. Where a setting delays the state-level terms (Lean: zero for the first 20% of training and ramped over the next 10%; reversible programs: delay 800 and ramp 400 of 16,000 steps; addition and counting: 400 and 200; cube: 800 and 400), the reason is an order-blind phase: while a tracker is still blind to event order, the equivalence term pulls its code towards the abelianisation of the task's group, which on S_5 is the sign of the permutation [45].

Selection, evaluation and statistics

We register numeric predictions in version control before every counted run; the ledger in the Supplementary Information lists every outcome, misses included. Checkpoints are selected by validation at the training length, after every term is on, never at test lengths; S_5 from scratch uses the final weights, as its registered protocol fixed, and has no delay, because the state-level term did not slow the formation of its class codes. Test sets are fixed per length with 512 problems drawn from the training distribution. A run is exact at a length if its last-event accuracy is at least 0.99; a problem is never wrong if the readout is right after every event. Intervals on counts are exact Clopper–Pearson 95% intervals (at 10 of 10 seeds, 0.69–1.00); noise thresholds carry bootstrap intervals over seeds. Drop-one comparisons use one-sided Fisher exact tests, which treat seeds as unpaired, with Holm correction over the 22 tests of this kind, the drop-one tests and M against our earlier recipe on S_5.

Top Lyapunov factor

We run power iteration of the closed tangent map along 16 random sequences of 512 events, discard the first 64 events, and exponentiate the mean log growth per event.

Balls

For a class q, the centre c_q is the normalised mean view of the class over random sequences, and r_q is the distance from c_q to the nearest facet of its readout cell, which is a polyhedron because the readout is affine in the view under the centred closure; (O_r) holds by construction. One-event and block images are computed for points on the ball's boundary, in random tangent directions and along the top direction, and containment is checked with true distances in the full space. In the S_5 tracker drawn in Fig. 2 the radii run from 0.163 to 0.228. The median per-event miss is 0.39 of the

median radius for the tracker drawn (0.28–0.46 across 20 seeds). The perturbation moves the view by a chord of 0.9 radii along the top direction mid-sequence.

Noise

After every event, $w \leftarrow \mathcal{N}(w + \varepsilon\,|c(w)|\,\xi)$, with $c(w) = w - \bar{w}\mathbf{1}$ the centred state ($\bar{w}$ its mean coordinate), ξ isotropic in the centred subspace with $\mathbb{E}|\xi|^2 = 1$, and $\mathcal{N}$ the closure, so ε is relative to the state's centred norm. ε_{99} and ε_{50} are the noise sizes at which exactness at 1,024 events falls to 0.99 and one half; a seed not exact without noise has no ε_{99}, and we report medians with and without imputing zero for it. Because the noise has unbounded support, we report rates at this length only [46].

Certificate

The snapped tracker replaces the state after every event by the nearest class code. Its transitions then form a finite graph, which breadth-first search checks on every edge and rendering. For the continuous trackers we bound the joint spectral radius of the error dynamics at the class centres over all blocks of two to four events, and run 3,840 repeated rotations per seed (a rewrite and its inverse at one position, to long lengths from real prefixes).

Frontier models

The models answered without tools at a budget of 64,000 output tokens including thinking, and at their largest budgets for some cells (128,000 tokens for Claude Fable 5.1 and GPT-6 Astra; a long decode for Gemini 4 Argon), 1 to 5 problems per model and cell; the Supplementary Information gives every cell's count. A miss is a wrong final answer or a budget stop; we report which. Over all cells but the cube, 7 of the 34 misses are budget stops; the Supplementary Information gives the prompts.

In-model step

L-TF trains a linear step on the frozen model's layer-12 code (width 896), initialised by least squares and teacher-forced to the true code with a readout loss and a closure per slot, and tests it running free from the start code. L-F resumes from it and adds F (ramp 1,000 steps, 14 power-iteration steps), under a registered rule that stops a seed if its free-running fraction drops by more than 0.02.

Hardware and reproducibility

All counted runs ran on cloud containers. Every record stores the host's instruction-set flags (AVX-512, AVX2) and a hash of its CPU flags; we do not claim bit reproducibility across hosts. Every number in this paper is a macro generated by `numbers.py` from the run records, each with its source path, and `AUDIT.md` maps every claim to its record or proof.

Proofs

Theorem 1(a): around each of the finitely many reached views, add a bump function times the difference between the chosen and the actual Jacobian, applied to the offset from the view, and close again; values at reached views are unchanged and the Jacobians there are the chosen ones. (b): the gradient of ℓ_τ in z is $\sum_{j\neq y} p_j(c_j - c_y)$, of norm at most $2s$, and the Hessian is $\tau^{-1}\mathrm{Cov}_{J\sim p}(c_J) \preceq (4s/\tau)I$; Jensen's inequality with $\sum_j c_j = 0$ bounds ℓ_τ between $-\langle z, c_y\rangle$ and $-\langle z, c_y\rangle + e^{1/\tau}/(2\tau)$. (c): by non-expansion and induction the state error after t events is at most εt; each logit difference then moves by at most $a = \sqrt{2}K\varepsilon t$, so the cross-entropy rises by at most $\log(1+s(e^a-1)) \le s(e^a-1)$, and averaging over positions gives the bound; the readout stays right while $\sqrt{2}K\varepsilon N < m$. On a line with logit gap $m - \sqrt{2}Kx$, the translation by ε is non-expansive and fails exactly when $\sqrt{2}K\varepsilon N > m$. Without non-expansion the bound fails: a map with derivative two near the exact views grows the error as $\varepsilon(2^t - 1)$. Corollary 1: a bump supported in a small ball that avoids every point the objective reads leaves every value and derivative it reads unchanged, adaptive reads included, and can move the chosen point anywhere on the sphere; for a reached view, the ball also avoids the earlier views of the shortest input that reaches it, so that input still reaches it. Theorem 2: after the prefix, a view in $B(c_q, r_q)$ at a block boundary stays in readout cells during the block by (C_r) and returns to $B(c_{q\cdot w}, \lambda r_{q\cdot w})$ at the next boundary; a perturbation of size $(1-\lambda)r$ there keeps it in $B(c_{q\cdot w}, r_{q\cdot w})$, so the first boundary that may be perturbed is $p+k$, and the readout is right at boundaries by (O_r). Lemma 1: with $E = \bar{z} - z(v)$, the two images minus $z(v)$ are $E \pm D$ and the cross terms cancel. Full proofs are in the Supplementary Information. They were drafted with the language models named below. Theorem 1, Corollary 1, Theorem 2 and Lemma 1 are machine-checked in Lean 4 with Mathlib, with the hypotheses stated here; the Supplementary Information names the Lean theorem beside each statement and marks the steps proved only on paper.

Use of large language models

Language models did much of this work under the author's direction. Anthropic's Claude models, used directly and through Cursor's agents, wrote most of the code, ran the registered experiments on cloud hardware, drafted the proofs and drafted this manuscript. The core theorems are machine-checked in Lean 4 with Mathlib: all three parts of Theorem 1, with the sharp construction, the resolution bound and the counterexample without non-expansion; Corollary 1; Theorem 2, and the failure of its converse on a line; Lemma 1; and, in the Supplementary Information, the enumeration, the theorem that a certificate must read the readout, the snap, the clip, the certificate and the radius, as stated there. The formalisation was written with the same models and is not a substitute for the author's responsibility for the proofs. The author chose the questions, approved every registered prediction and every claim, and is responsible for the content. Predictions were committed to version control before each counted run; every number in the text is generated from run records by a script; and an audit file maps each claim to its record or proof. GPT-6 Astra and Gemini 4 Argon appear only as test subjects. Every figure is drawn by code from the records; no image in this

paper was generated by an AI model, and images generated while exploring layouts appear nowhere in it.

Data availability. Every run record, registered prediction and analysis script behind the numbers in this paper will be released with it at a public repository (address to be added). `numbers.py` regenerates every number from the records.

Code availability. The training, evaluation, certificate and figure code will be released at the same repository (address to be added).

Supplementary information. Full proofs, the repeated-word sweeps, stories and programs, the multiplicity table, the frontier cells, the settings in detail, and the ledger of registered predictions with their outcomes, misses included.

Declarations

Competing interests. The author declares no competing interests. **Funding.** This work was self-funded.

References

[1] Sinha, A., Arun, A., Goel, S., Staab, S. & Geiping, J. The Illusion of Diminishing Returns: Measuring Long Horizon Execution in LLMs. Preprint at https://arxiv.org/abs/2509.09677 (2025). arXiv:2509.09677.

[2] Yen, H., Mani, N., Renda, A. & Hardin, C. S. LEAPS: Long-context Evaluation of Adaptive Pattern Learning and State Tracking. Preprint at https://arxiv.org/abs/2610.25919 (2026). arXiv:2610.25919.

[3] Guo, D., Wu, J. & Yiu, S. M. The Deterministic Horizon: When Extended Reasoning Fails and Tool Delegation Becomes Necessary. Preprint at https://arxiv.org/abs/2606.00376 (2026). arXiv:2606.00376.

[4] Delétang, G. *et al. Neural Networks and the Chomsky Hierarchy. International Conference on Learning Representations* (2023). URL https://arxiv.org/abs/2207.02098. arXiv:2207.02098.

[5] Liu, B., Ash, J. T., Goel, S., Krishnamurthy, A. & Zhang, C. *Transformers Learn Shortcuts to Automata. International Conference on Learning Representations* (2023). URL https://arxiv.org/abs/2210.10749. arXiv:2210.10749.

[6] Merrill, W., Petty, J. & Sabharwal, A. *The Illusion of State in State-Space Models. Proceedings of the 41st International Conference on Machine Learning* (2024). URL https://arxiv.org/abs/2404.08819. arXiv:2404.08819.

[7] Grazzi, R. *et al. Unlocking State-Tracking in Linear RNNs Through Negative Eigenvalues. International Conference on Learning Representations* (2025). URL https://arxiv.org/abs/2411.12537. arXiv:2411.12537.

[8] Siems, J. *et al. DeltaProduct: Improving State-Tracking in Linear RNNs via Householder Products. Advances in Neural Information Processing Systems* (2025). URL https://arxiv.org/abs/2502.10297. arXiv:2502.10297.

[9] Jim, K.-C., Giles, C. & Horne, B. An analysis of noise in recurrent neural networks: convergence and generalization. *IEEE Transactions on Neural Networks* **7**, 1424–1438 (1996). URL https://doi.org/10.1109/72.548170.

[10] Oliva, C. & Lago-Fernández, L. F. Stability of Internal States in Recurrent Neural Networks Trained on Regular Languages. *Neurocomputing* **452**, 212–223 (2021). URL https://doi.org/10.1016/j.neucom.2021.04.058.

[11] Das, S. & Mozer, M. Dynamic On-line Clustering and State Extraction: An Approach to Symbolic Learning. *Neural Networks* **11**, 53–64 (1998). URL https://doi.org/10.1016/S0893-6080(97)00113-5.

[12] Mozer, M. C., Kazakov, D. & Lindsey, R. V. State-Denoised Recurrent Neural Networks. Preprint at https://arxiv.org/abs/1805.08394 (2018). arXiv:1805.08394.

[13] Chung, J., Choi, H. & Kim, S. J. Rethinking State Tracking in Recurrent Models Through Error Control Dynamics. Preprint at https://arxiv.org/abs/2605.07755 (2026). arXiv:2605.07755.

[14] Wei, Z. On the Spatiotemporal Dynamics of Generalization in Neural Networks. Preprint at https://arxiv.org/abs/2602.01651 (2026). arXiv:2602.01651.

[15] Nguyen, T.-K. *et al. ControlJEPA: Principled Trajectory Regularization via Lyapunov Tube Loss. Advances in Neural Information Processing Systems* (2026). URL https://arxiv.org/abs/2610.24951. arXiv:2610.24951.

[16] Maass, W. & Orponen, P. On the Effect of Analog Noise in Discrete-Time Analog Computations. *Neural Computation* **10**, 1071–1095 (1998). URL https://doi.org/10.1162/089976698300017359.

[17] Casey, M. The Dynamics of Discrete-Time Computation, with Application to Recurrent Neural Networks and Finite State Machine Extraction. *Neural Computation* **8**, 1135–1178 (1996). URL https://doi.org/10.1162/neco.1996.8.6.1135.

[18] Lan, N., Chemla, E. & Katzir, R. *Bridging the Empirical-Theoretical Gap in Neural Network Formal Language Learning Using Minimum Description Length. Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 13198–13210 (Association for Computational Linguistics, 2024). URL https://doi.org/10.18653/v1/2024.acl-long.713. arXiv:2402.10013.

[19] Vafa, K., Chen, J. Y., Rambachan, A., Kleinberg, J. & Mullainathan, S. *Evaluating the World Model Implicit in a Generative Model. Advances in Neural Information Processing Systems* (2024). URL https://arxiv.org/abs/2406.03689. arXiv:2406.03689.

[20] Myhill, J. Finite Automata and Representation of Events. in *Fundamental Concepts in the Theory of Systems, WADC Technical Report 57-624* 112–137 (Wright Air Development Center, 1957).

[21] Nerode, A. Linear automaton transformations. *Proceedings of the American Mathematical Society* **9**, 541–544 (1958). URL https://doi.org/10.1090/S0002-9939-1958-0135681-9.

[22] Brandoit, J. *et al.* Length-Independent State Tracking Under a Parallel Scan. Preprint at https://arxiv.org/abs/2609.32447 (2026). arXiv:2609.32447.

[23] Bishop, C. M. Training with Noise is Equivalent to Tikhonov Regularization. *Neural Computation* **7**, 108–116 (1995). URL https://doi.org/10.1162/neco.1995.7.1.108.

[24] Rifai, S., Vincent, P., Muller, X., Glorot, X. & Bengio, Y. *Contractive Auto-Encoders: Explicit Invariance During Feature Extraction. Proceedings of the 28th International Conference on Machine Learning* (2011). URL https://icml.cc/2011/papers/455_icmlpaper.pdf.

[25] Miyato, T., Maeda, S.-i., Koyama, M. & Ishii, S. Virtual Adversarial Training: A Regularization Method for Supervised and Semi-Supervised Learning. *IEEE Transactions on Pattern Analysis and Machine Intelligence* **41**, 1979–1993 (2019). URL https://doi.org/10.1109/TPAMI.2018.2858821.

[26] Revay, M. & Manchester, I. R. Contracting Implicit Recurrent Neural Networks: Stable Models with Improved Trainability. Preprint at https://arxiv.org/abs/1912.10402 (2019). arXiv:1912.10402.

[27] Erichson, N. B., Azencot, O., Queiruga, A., Hodgkinson, L. & Mahoney, M. W. *Lipschitz Recurrent Neural Networks. International Conference on Learning Representations* (2021). URL https://arxiv.org/abs/2006.12070. arXiv:2006.12070.

[28] Kozachkov, L., Ennis, M. & Slotine, J.-J. *RNNs of RNNs: Recursive Construction of Stable Assemblies of Recurrent Neural Networks. Advances in Neural Information Processing Systems* (2022). URL https://arxiv.org/abs/2106.08928. arXiv:2106.08928.

[29] Hammer, B. & Tiňo, P. Recurrent Neural Networks with Small Weights Implement Definite Memory Machines. *Neural Computation* **15**, 1897–1929 (2003). URL https://doi.org/10.1162/08997660360675080.

[30] Hui, B. *et al.* Qwen2.5-Coder Technical Report. Preprint at https://arxiv.org/abs/2409.12186 (2024). arXiv:2409.12186.

[31] Qwen Team. Qwen2.5 Technical Report. Preprint at https://arxiv.org/abs/2412.15115 (2024). arXiv:2412.15115.

[32] Barrington, D. A. Bounded-width polynomial-size branching programs recognize exactly those languages in NC1. *Journal of Computer and System Sciences* **38**, 150–164 (1989). URL https://doi.org/10.1016/0022-0000(89)90037-8.

[33] Merrill, W. & Sabharwal, A. The Parallelism Tradeoff: Limitations of Log-Precision Transformers. *Transactions of the Association for Computational Linguistics* **11**, 531–545 (2023). URL https://doi.org/10.1162/tacl_a_00562.

[34] Merrill, W. & Sabharwal, A. *The Expressive Power of Transformers with Chain of Thought. International Conference on Learning Representations* (2024). URL https://arxiv.org/abs/2310.07923. arXiv:2310.07923.

[35] Giles, C. L. *et al.* Learning and Extracting Finite State Automata with Second-Order Recurrent Neural Networks. *Neural Computation* **4**, 393–405 (1992). URL https://doi.org/10.1162/neco.1992.4.3.393.

[36] Omlin, C. W. & Giles, C. Extraction of rules from discrete-time recurrent neural networks. *Neural Networks* **9**, 41–52 (1996). URL https://doi.org/10.1016/0893-6080(95)00086-0.

[37] Weiss, G., Goldberg, Y. & Yahav, E. *Extracting Automata from Recurrent Neural Networks Using Queries and Counterexamples. Proceedings of the 35th International Conference on Machine Learning* (2018). URL https://arxiv.org/abs/1711.09576. arXiv:1711.09576.

[38] Kleene, S. C. Representation of Events in Nerve Nets and Finite Automata. in *Automata Studies (AM-34)* 3–42 (Princeton University Press, 1956). URL https://doi.org/10.1515/9781400882618-002.

[39] von Neumann, J. Probabilistic Logics and the Synthesis of Reliable Organisms From Unreliable Components. in *Automata Studies (AM-34)* 43–98 (Princeton University Press, 1956). URL https://doi.org/10.1515/9781400882618-003.

[40] Pippenger, N. *On networks of noisy gates. 26th Annual Symposium on Foundations of Computer Science (sfcs 1985)*, 30–38 (IEEE, 1985). URL https://doi.org/10.1109/SFCS.1985.41.

[41] Pollack, J. B. The induction of dynamical recognizers. *Machine Learning* **7**, 227–252 (1991). URL https://doi.org/10.1007/BF00114845.

[42] Zeng, Z., Goodman, R. M. & Smyth, P. Learning Finite State Machines With Self-Clustering Recurrent Networks. *Neural Computation* **5**, 976–990 (1993). URL https://doi.org/10.1162/neco.1993.5.6.976.

[43] de Moura, L. & Ullrich, S. *The Lean 4 Theorem Prover and Programming Language. Automated Deduction – CADE 28, Lecture Notes in Computer Science*, 625–635 (2021). URL https://doi.org/10.1007/978-3-030-79876-5_37.

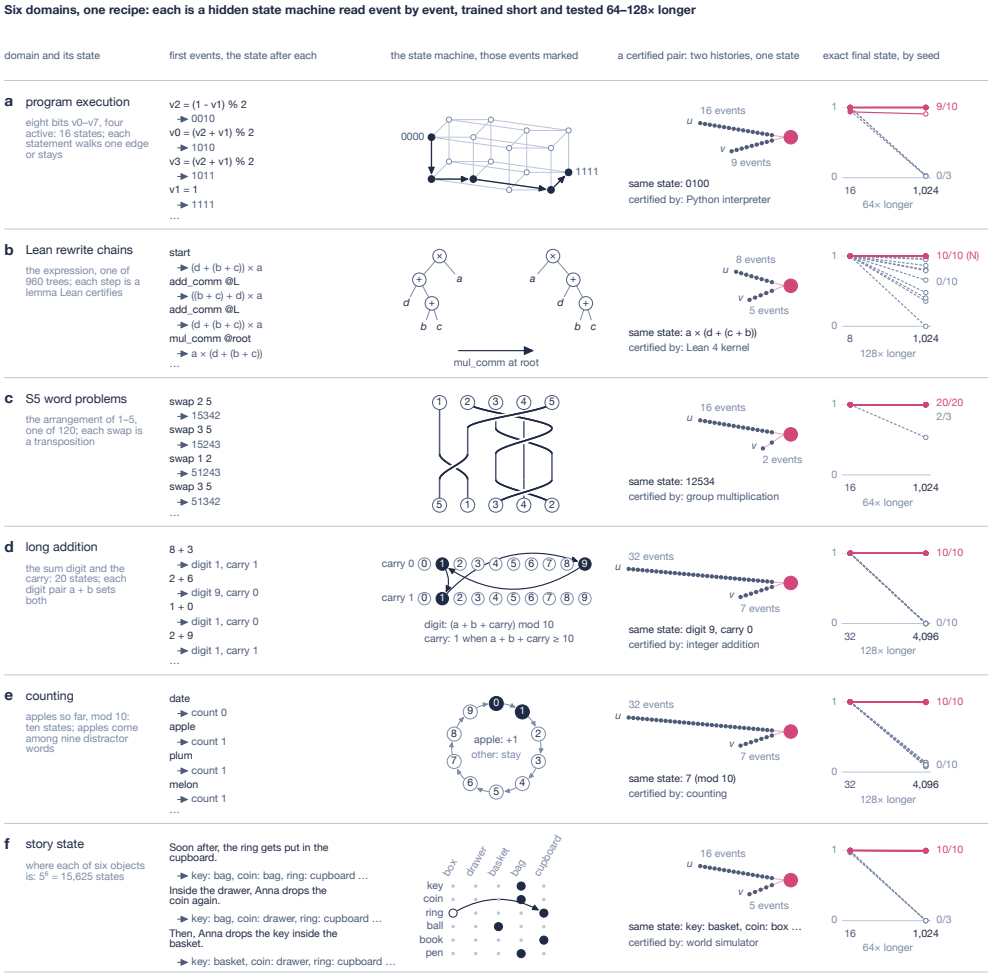
[44] Loshchilov, I. & Hutter, F. *Decoupled Weight Decay Regularization. International Conference on Learning Representations* (2019). URL https://arxiv.org/abs/1711.05101. arXiv:1711.05101.

[45] Zhang, Z. & Li, Y. Tracking States or Tracking Cosets? An Algebraic Account of Learned State Tracking. Preprint at https://arxiv.org/abs/2609.29951 (2026). arXiv:2609.29951.

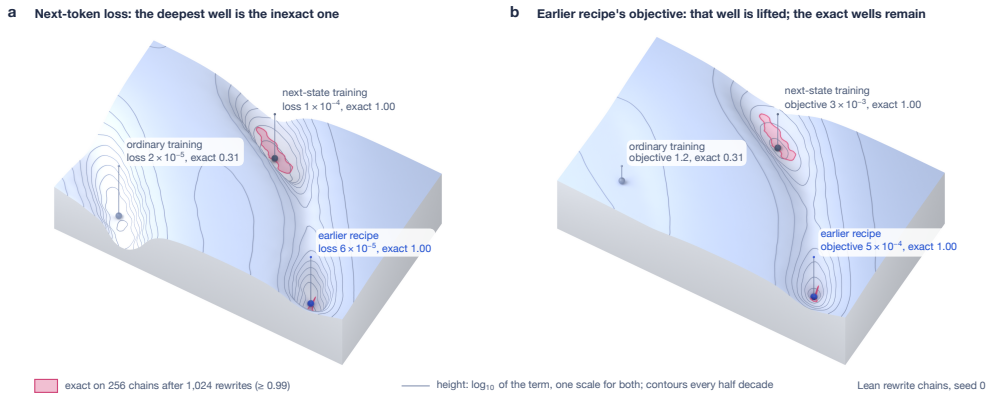
[46] Maass, W. & Sontag, E. D. Analog Neural Nets with Gaussian or Other Common Noise Distributions Cannot Recognize Arbitrary Regular Languages. *Neural Computation* **11**, 771–782 (1999). URL https://doi.org/10.1162/089976699300016656.

Extended Data

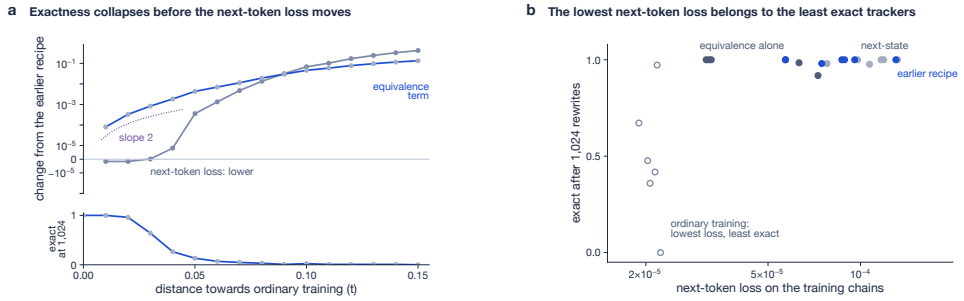
1. The settings: events, states, a certified pair and every seed's exactness (Extended Data Fig. 1).
2. The landscape of next-token loss against the earlier recipe's objective, an illustration (Extended Data Fig. 2).
3. The next-token loss along the weight line (Extended Data Fig. 3).
4. The learned state condenses onto the automaton (Extended Data Fig. 4).
5. The terms and the theorem's clauses (Extended Data Fig. 5).
6. Every objective in every setting, closure-only baselines included (Extended Data Fig. 6).
7. Frontier models against our tracker, with output tokens per problem (Extended Data Fig. 7).
8. The Rubik's cube tracker (Extended Data Fig. 8).
9. Protocols, the registration ledger and the extended tables are in the Supplementary Information.



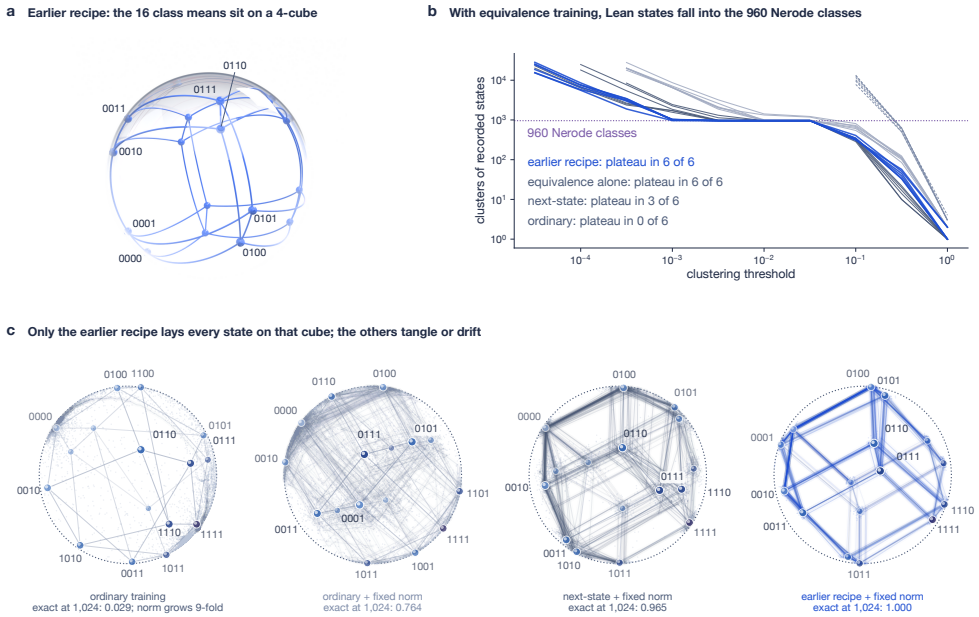
Extended Data Fig. 1 The settings. Per setting, the first events and states, the hidden state machine with those events marked, one certified pair, and every seed's exactness at the training and test lengths under M (rose) and ordinary training (grey). **a,** Eight-variable programs (Supplementary Information), not the reversible programs of the A_{16} word problem. **b,** Lean rewrite chains. **c,** S_5 words from scratch; the ordinary-training count comes from the separate study of Table 1. **d,** Addition: the tracker's state is the pair of sum digit and carry, one row of states per carry. **e,** Counting. **f,** Stories.



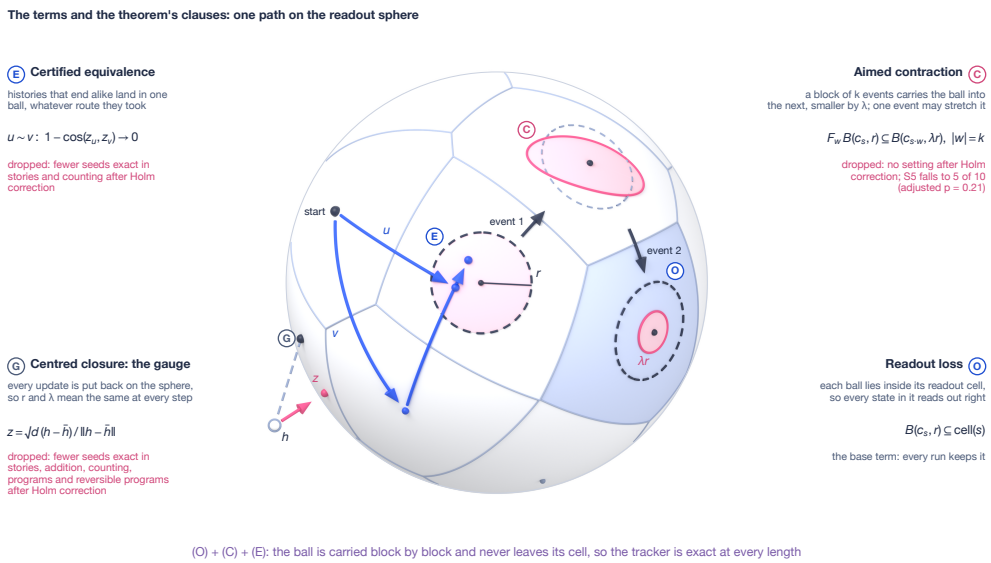
Extended Data Fig. 2 The landscape, an illustration. Next-token loss against the earlier recipe's objective, two terrains on one height scale (Lean). Interpolations between trained networks generally show a well at each end; the earlier recipe's objective lifts the inexact well and keeps the exact ones. The trackers optimise different objectives, so this is not a comparison at matched cross-entropy.



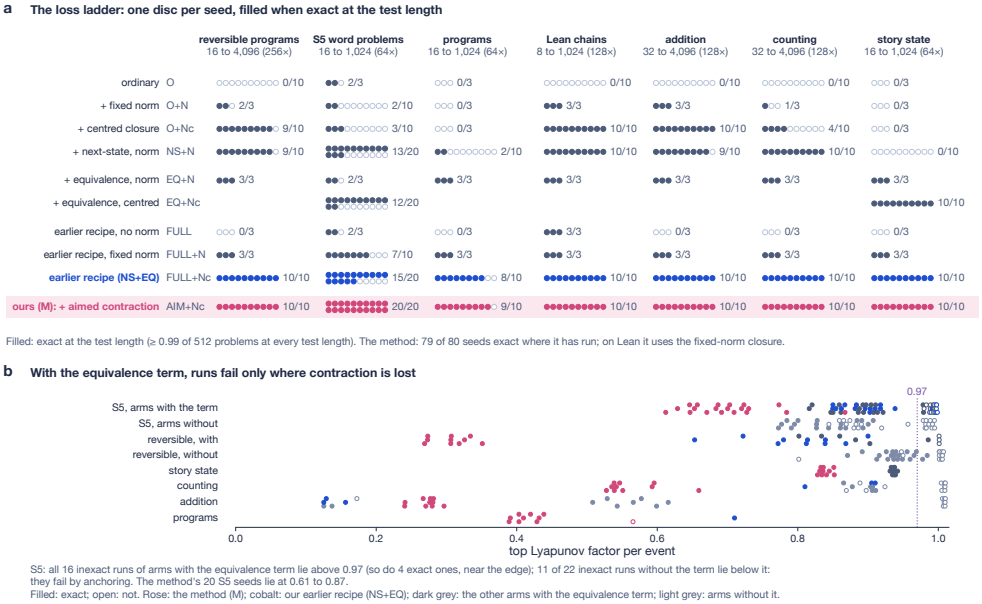
Extended Data Fig. 3 The next-token loss along the weight line. a, Moving from our earlier recipe towards ordinary training along the weight line, exactness collapses while the next-token loss barely moves and the equivalence term rises from the first step. **b,** The trackers with the lowest next-token loss are the least exact; the exact trackers also optimise terms that compete with cross-entropy, so this is an observation, not evidence for Theorem 1.



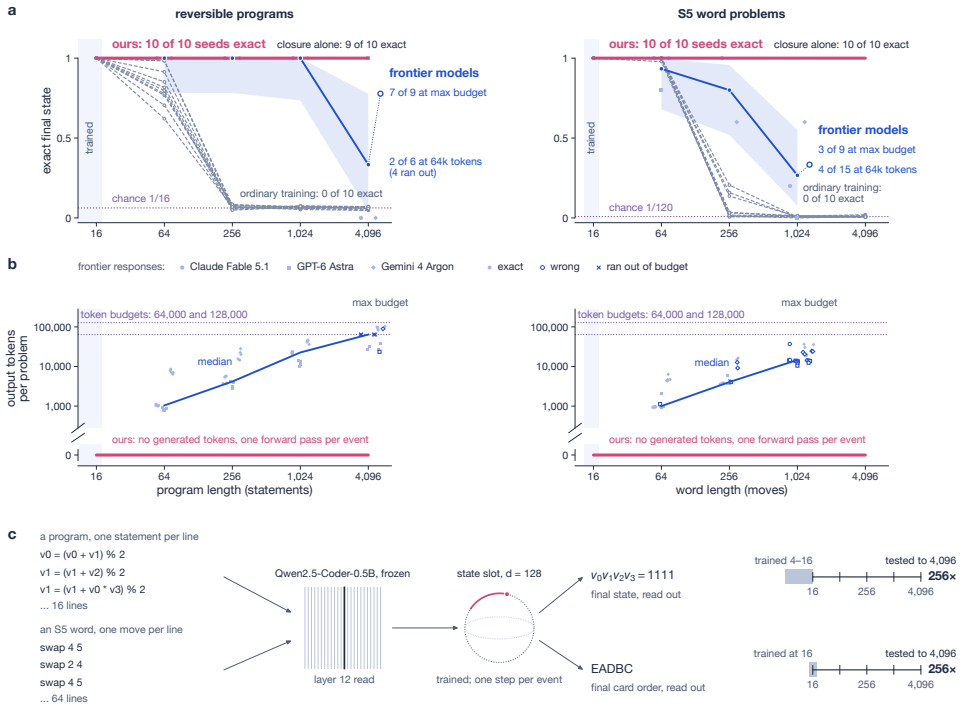
Extended Data Fig. 4 The learned state condenses onto the automaton. a, Class means of a program tracker trained with the earlier recipe sit on a four-dimensional cube. **b,** With equivalence training, Lean states fall into the Nerode classes. Next-state training reaches that plateau on some of its seeds; ordinary training does not. **c,** Only the earlier recipe lays every state on that cube.



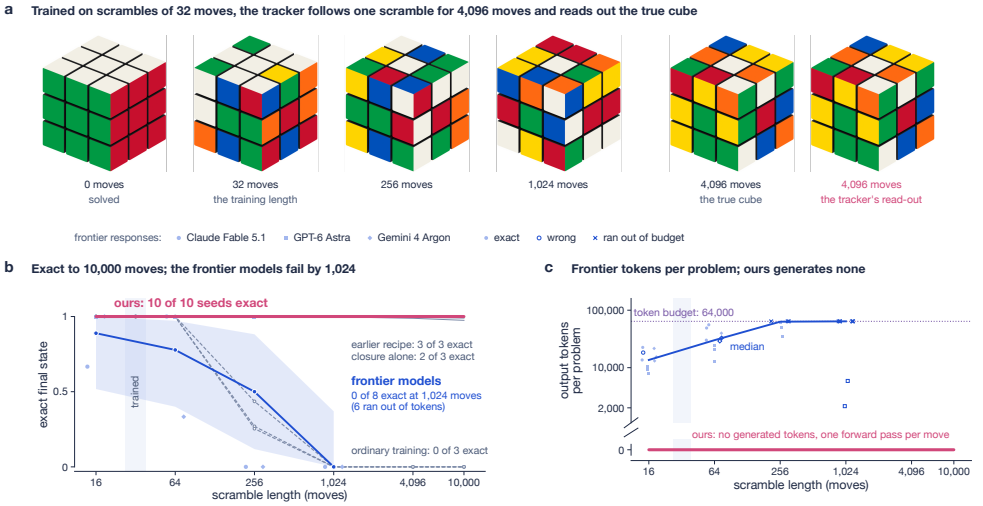
Extended Data Fig. 5 The terms and the theorem's clauses. One path on the readout sphere: two equivalent histories land in one ball (E_r); a block of events carries the ball into a smaller copy of the next, though one event may stretch it out of its ball while its states stay in their readout cells (C_r); the ball sits inside its readout cell (O_r); an update that leaves the sphere is put back by the closure. Each callout names the settings where dropping that term leaves fewer seeds exact after Holm correction, over the same family as Fig. 3.



Extended Data Fig. 6 Every objective in every setting. a, One disc per seed. The closure with a readout loss alone (O+Nc) is exact on every seed in addition (10) and Lean (10), on 9 of 10 in reversible programs and on 4 of 10 in counting; M is exact on 79 of 80 seeds. Certified equivalence with the centred closure (EQ+Nc) was run only on S_5 and stories, so its other cells are empty. The S_5 cell of ordinary training (O) comes from a separate study with its own protocol. **b,** The top Lyapunov factor per run, on an axis from zero.



Extended Data Fig. 7 Exact at any tested length, with no generated tokens. a, Exact final state against length on reversible programs and S_5 read as text: our trackers, the closure alone (9 of 10 and 10 of 10), ordinary training and the frontier models, pooled over 1 to 5 problems per model and length (Supplementary Information), with budget stops marked apart from wrong answers. **b,** Frontier output tokens per problem grow with length and reach the 64,000- and 128,000-token budgets; our trackers generate none. **c,** The shared architecture: events, one per line, into the frozen model, read at layer 12 into a state slot of width 128 that steps once per event.



Extended Data Fig. 8 The Rubik's cube. The factorised tracker on the frozen Qwen2.5-0.5B, trained on scrambles of 32 moves, is exact on all 10 seeds at 4,096 and 10,000 moves. At 10,000 the closure alone is exact on 2 of 3 seeds, the earlier recipe on 3 of 3, and ordinary training on 0 of 3. Panel b pools the frontier responses on record, marks budget stops apart from wrong answers, and some cells are short of a full set; the Supplementary Information gives every cell's count.

Supplementary Information: Closure and aimed perturbation keep neural state exact far beyond the training length

Neo [surname]

The main text states the theorems and the results. This note gives the proofs, the registration ledger with every registered outcome including the misses, the repeated-word sweeps, stories and programs, the multiplicity table, the protocols, the extended tables, the frontier cells, the recorded compute, and the language-model disclosure. Every number is generated from a run record. References are those of the main text.

1 Proofs

Views lie on the centred unit sphere. The code's closure keeps the centred state at norm $\sqrt{d}$, with d the state's width; the view is that state divided by its norm, and radii, perturbation sizes σ, noise sizes ε and the distances in the loss are chords between views. For a nonzero centred vector h, $\mathcal{N}_0(h) = h/|h|$ is the closure in these units (`normalize`). $d(a, b) = \frac{1}{2}|a - b|^2$ is the centred cosine distance. A tracker is closed when every event map sends views to views. Class codes c_j are centred, $\sum_j c_j = 0$. Theorems S1 and S3 and the readout calculation are Theorem 1 of the main text, Corollary S2 is its Corollary 1, Theorem S4 is its Theorem 2, and Lemma S1 is its lemma.

Each statement below is followed by the Lean theorems that formalise it, in the Lean 4 formalisation with Mathlib (`research/lean`, namespace `Polyformer`, commit `319c2f0c`). None of them uses `sorry`, and each depends only on the axioms `propext`, `Classical.choice` and `Quot.sound`. Steps that are not in Lean are marked as proved on paper.

1.1 Blindness (Theorem 1(a))

Theorem S1 (Blindness) *Let a closed tracker's views on the training data take finitely many values. For any linear maps between the tangent spaces at those views there is a smooth tracker, not necessarily one the network family can represent, that agrees with it at every reached view and has those maps as its Jacobians there. Every objective that reads the transition only through its values at reached views takes the same value at both trackers.*

Lean: `blindness`, for trackers of class C^n, $1 \le n \le \infty$, on an open set containing the sphere.

Proof Let F_x be the view map of an event and let the reached views be finitely many distinct points c_s. Choose disjoint balls and bump functions φ_s equal to one near each c_s. Near c_s set

$$F'_x(z) = \mathcal{N}_0\big(F_x(z) + \varphi_s(z)\,(M_{s,x} - DF_x(c_s))P_s(z - c_s)\big),$$

and $F'_x = F_x$ elsewhere, with P_s the orthogonal projection onto the tangent space at c_s and $M_{s,x}$ mapping that tangent space into the tangent space at $F_x(c_s)$. The added term vanishes at c_s, so the values agree. Its derivative at c_s is $M_{s,x} - DF_x(c_s)$, and the derivative of the closure at a unit centred vector is the identity on tangent vectors, so $DF'_x(c_s) = M_{s,x}$. An objective that reads only values at reached views cannot tell F from F'. □

Corollary S2 (Finite reads) *Let an objective read values, and finitely many derivatives, of the transition at finitely many points, possibly chosen from what it has already read. For any closed tracker with at least two readout cells, any radii r_q, any event x and any point h of a ball $B(c_q, r_q)$ that the objective does not read, there is a smooth closed tracker with the same objective value whose map F_x sends h out of the readout cell of $o(q \cdot x)$. Reached-state objectives and F are such objectives, so none of them certifies (C_r) for given radii. Likewise, if an input u reaches a view that the objective does not read and an event x is applicable after u, there is a smooth closed tracker with the same objective value that is not exact.*

Lean: `finite_reads` for the first statement, with any unread view in place of a point of a ball; `exists_tracker_failing_rmn` for the last, with the read points fixed. That F and reached-state objectives read in this way is checked on paper, below.

Proof The read points are finitely many and h is not among them, so a small ball U around h avoids them all. Choose a bump φ supported in U with $\varphi(h) = 1$ and set $F'_x = \mathcal{N}_0\big((1 - \varphi)F_x + \varphi\, u\big)$ for a view u outside the readout cell of $o(q \cdot x)$ that is not antipodal to any value of F_x on U. Outside U, and so at every read point together with all derivatives there, $F'_x = F_x$. An adaptive objective chooses its next point from values it has read, so it reads the same points and the same values from both trackers. For the last statement, let u_0 be the shortest prefix of u that reaches the unread view h, and let y be the event that follows u_0 in u, or x if $u_0 = u$; then y is applicable after u_0. Shrink U so that it also avoids the earlier views of the run of u_0, which differ from h, and move F_y in place of F_x, to a view outside the readout cell of the state after $u_0 y$. The new tracker still reaches h by u_0, and its readout after $u_0 y$ is wrong. F is such an objective: it reads values at the reached views and at two perturbed views per training history, and, through the forward direction ψ, first derivatives along the run. Nerode training is therefore motivated by the clauses of Theorem S4, not derived from them. □

1.2 The readout (Theorem 1(b)) and likelihood

At every temperature $\tau > 0$, the gradient of the readout loss in the view is $\sum_{j \neq y} p_j(c_j - c_y)$, of norm at most $2s$, and the Hessian is $\tau^{-1}\mathrm{Cov}(c_J) \preceq (4s/\tau)I$. Both carry the error probability s; this is the familiar saturation of cross-entropy that margin losses address. When the head's rows have equal norm and no bias, the head's logits are $\langle z, c_j \rangle / \tau$ with τ set by that norm, so next-token likelihood on the closed

view is the readout loss at that temperature, up to the factor $1/\tau$ and the constant $\log C$. With the layer norm before the head, which is affine on the centred sphere, the rows and the bias are the effective ones, with its gain and bias folded in. With unequal row norms or a bias the identification does not hold.
Lean: `norm_gradient_readoutLoss_le`, `hessian_readoutLoss_le`, `tendstoUniformlyOn_readoutLoss` and `readoutLoss_bounds` for the bounds and for the high-temperature limit in Theorem 1(b) of the main text; `likelihood_is_blind`, `readoutLikelihood_eq` and `fderiv_run_eq_tangentRun` for the consequence of Theorem S1 that the next-token loss, the certified-equivalence term and next-state prediction take the same value at both trackers, while the derivative of the word map along every reached path is the composition of the prescribed maps.

1.3 The excess loss (Theorem 1(c))

Theorem S3 (Excess loss) *Let the exact tracker have views z_t and the learned one views $\hat{z}_t$, from the same start. Suppose every learned event map $\hat{F}_x$ is non-expansive on a set that contains every z_t and $\hat{z}_t$, and misses the exact map by at most ε at exact views, $|\hat{F}_x(z_t) - F_x(z_t)| \le \varepsilon$. Let the logit vector be K-Lipschitz in the view (Euclidean norms), let m be the exact tracker's smallest logit margin, and let $\bar{s}$ be its mean error probability over the loss's positions $t \le L$. Then:* (i) $|\hat{z}_t - z_t| \le \varepsilon t$; (ii) *the mean cross-entropy over those positions exceeds the exact tracker's by at most $\bar{s}\,(e^{\sqrt{2}K\varepsilon L} - 1)$;* (iii) *the readout is right at every $t \le N$ when $\sqrt{2}K\varepsilon N < m$;* (iv) *the constant is sharp: some non-expansive transition with miss ε fails at every $N > m/(\sqrt{2}K\varepsilon)$;* (v) *for a loss resolved to ρ, the transition of (iv) with $\varepsilon = \log(1+\rho/\bar{s})/(\sqrt{2}KL)$ lies within ρ of the exact loss and fails at every $N > mL/\log(1+\rho/\bar{s})$.*

Lean: (i) `norm_run_sub_run_le`; (ii) `excess_loss_le`, for any non-negative weights over positions up to L, with `excess_crossEntropy_le` at one position; (iii) `readout_correct_of_lt_margin`; (iv) `theoremA_sharp`, on the mirror-image drift $x \mapsto x - \varepsilon$; (v) `theoremA_resolution`. Views may lie in any normed space.

Proof (i) $|\hat{z}_{t+1} - z_{t+1}| \le |\hat{F}(\hat{z}_t) - \hat{F}(z_t)| + |\hat{F}(z_t) - F(z_t)| \le |\hat{z}_t - z_t| + \varepsilon$, and induction.

(ii) Let $a_t = \sqrt{2}K\varepsilon t$. For a wrong label j, the gap $\ell_y - \ell_j$ changes by at most $|\Delta\ell_y| + |\Delta\ell_j| \le \sqrt{2}\,|\Delta\ell| \le \sqrt{2}K|\hat{z}_t - z_t| \le a_t$. The cross-entropy is $\log(1+S)$ with $S = \sum_{j\neq y} e^{\ell_j - \ell_y}$. Each term of S grows by at most the factor e^{a_t}, so the cross-entropy rises by at most $\log\big((1+e^{a_t}S)/(1+S)\big) = \log\big(1 + s_t(e^{a_t}-1)\big) \le s_t(e^{a_t}-1)$, where $s_t = S/(1+S)$ is the exact tracker's error probability at position t. Averaging over positions and using $a_t \le a_L$ gives (ii).

(iii) Every gap starts at least m and falls by at most $a_t \le \sqrt{2}K\varepsilon N < m$.

(iv) On a line, let the logits be $(\ell_y, \ell_j) = (m/2 - Kx/\sqrt{2},\ -m/2 + Kx/\sqrt{2})$, which are K-Lipschitz in x with gap $m - \sqrt{2}Kx$. Let the exact view stay at $x = 0$ and the learned map be $x \mapsto x + \varepsilon$, an isometry that misses by ε. After N events the gap is $m - \sqrt{2}K\varepsilon N$, negative exactly when $N > m/(\sqrt{2}K\varepsilon)$.

(v) Substitute this ε into (ii) and (iv). □

The transition of (iv) lowers the margin as fast as the readout allows. A non-expansive drift with the same miss ε whose margin falls by $\gamma^{-1}\sqrt{2}K\varepsilon$ per event, $\gamma \ge 1$,

fails only beyond $N > \gamma m/(\sqrt{2}K\varepsilon)$, which at the ε of (v) is $\gamma\, mL/\log(1+\rho/\bar{s})$; the main text's threshold is the steepest case, $\gamma = 1$. The drift statement is proved on paper. Non-expansion cannot be dropped: if the exact view is fixed at 0 and the learned map is $x \mapsto 2x+\varepsilon$, the miss is ε and the error after t events is $\varepsilon(2^t-1)$, so (i) fails; for suitable constants (iii) fails, which `theoremA_false_without_nonexpansion` checks on an instance, and a direct computation on that instance shows that (ii) fails as well.

The bound is what the loss can see, not a law for where a model fails. A model that does not fit its training length can fail earlier, and a model can be exact past this N for reasons the loss does not see.

1.4 Robust Myhill–Nerode (Theorem 2)

Theorem S4 (Robust Myhill–Nerode) *Let a closed tracker act by F_x, with codes c_q, radii $r_q > 0$, a block length $k \geq 1$ and a rate $\lambda \in [0,1)$. Suppose (O_r) every ball $B(c_q, r_q)$ lies in the readout cell of $o(q)$; (C_r) for every state q and applicable word w, $F_w(B(c_q, r_q))$ lies in the readout cell of $o(q \cdot w)$ if $|w| < k$, and in $B(c_{q\cdot w}, \lambda r_{q\cdot w})$ if $|w| = k$; (E_r) for some prefix length $p \geq 0$, the readout is right on every word shorter than p, and the view reached by every word u of length p lies in $B(c_{q_0\cdot u}, r_{q_0\cdot u})$. Then the readout is exact on every input at every length. It stays exact along any run that is unperturbed through the prefix, follows the tracker inside blocks, and is moved by at most $(1-\lambda)\min_q r_q$ at each block boundary $p+k, p+2k, \ldots$ while staying a view.*

Lean: `robustMyhillNerode_full`, with closed balls, partial automata and any pseudometric space of views; the perturbation may depend on the whole history. The view at p itself may not be moved: (E_r) places it in the ball, not in the smaller one.

Proof Words shorter than p are covered by (E_r). Cut a longer word after its prefix into blocks of k events and a final partial block. At the end of the prefix the view lies in the ball of its state, by (E_r). If the view at a block boundary lies in $B(c_q, r_q)$, then by (C_r) every view inside the block lies in the right readout cell, and the view at the next boundary lies in $B(c_{q\cdot w}, \lambda r_{q\cdot w})$, inside $B(c_{q\cdot w}, r_{q\cdot w})$, where the readout is right by (O_r). A perturbation of size at most $(1-\lambda)r_{q\cdot w}$ at that boundary keeps the view in $B(c_{q\cdot w}, r_{q\cdot w})$, and the induction continues from there; the first such boundary is $p+k$. The final partial block is covered by the first half of (C_r). □

What the theorem says. It is a sufficient condition and has no converse. The parity tracker $z \mapsto -z$ with the sign readout is exact, but its block maps are isometries, so (C_r) would need $r_q \leq \lambda r_{q\cdot w}$ for every q, which fails at the largest radius (`rmn_no_converse`, on a line; on the circle the same argument applies to caps, proved on paper). With closed balls of radius zero and $k = 1$, the clauses say that $q \mapsto c_q$ realises the task's minimal automaton inside the tracker, $F_x(c_q) = c_{q\cdot x}$ (`myhillNerode`; that the map is injective for the minimal automaton is proved on paper); minimality of the tracker is not a consequence. The closest prior statement, cited in the main text, is that length-independent tracking at finite precision is equivalent to a partition whose cells every event carries into the next with a margin. The block form replaces that per-event margin with a block length and a rate λ, and asks only that the views inside a block stay in their readout cells.

What the trained trackers satisfy. The ball study registered seven predictions on one trained S_5 tracker (M, seed 5; registration `1755ebe`) before measuring it, and four of them missed. The median radius is 0.187, below the registered range [0.25, 0.75]. The median per-event miss is 0.39 of the median radius against a registered 0.35, and the largest is 0.85 of the target radius against 0.8. One event carries a fraction 0.799 of 13,200 sampled boundary points into the next ball, against a registered 0.95: in random directions nearly all land inside, along the top singular direction almost none (0.998 and 0.002), with a median stretch of 1.7 radii. A block of 4 events has a median ratio of 0.242 along the top direction at a small chord, inside its registered band [0.15, 0.45], and carries a fraction 0.998 of boundary points inside the next ball, but the boundary images reach 0.88 radii at the 99th percentile and 1.40 at most, and only a fraction 0.82 are within the registered 0.6 radii. Every final state of 512 at every tested length lies in its class's ball, and a perturbation of the state falls to a median 0.072 of its size within 8 events (registered: below 0.1; upper quartile 0.127). Two unregistered diagnostics bear on the clauses as stated above. Every sampled boundary image decodes correctly after every event inside the block (fraction 1.000), although only 0.953 lie inside the current ball after the first event: the cell form of (C_r) holds on every sample and the ball form does not. The start view lies 4.6 radii from its ball, but the view after the first event lies inside its ball in 704 of 704 samples: (E_r) holds with prefix length one on every sample. No trained tracker is shown to satisfy (C_r) uniformly.

1.5 The one-event loss

Lemma S1 (Split) *Let $\bar{z}$ and D be the midpoint and the half-difference of the two perturbed images in the main text's loss, and $y = z(v)$. Then $\frac{1}{2}\sum_{s=\pm1} d(\bar{z}+sD, y) = \frac{1}{2}|\bar{z}-y|^2 + \frac{1}{2}|D|^2$.*

Lean: `kick_split`.

Proof With $E = \bar{z} - y$, the two images minus y are $E \pm D$, and the cross terms cancel. □

The first term merges the perturbed pair with the certified equivalent view. The second is the secant growth along ψ; the trained loss weights it by the gain G. If the bracket at $G = 1$ is at most τ, then $|\bar{z}-y| \le \sqrt{2\tau}$ and, because the perturbed chord has length $2\sigma/\sqrt{1+\sigma^2}$, the secant ratio is at most $\sqrt{2\tau}\,\sqrt{1+\sigma^2}/\sigma$ (`kick_chord`, `kick_secant_le`).

Growth along ψ. Along a run, write $g_t = |J_{t+1}\psi_t|$ and $\psi_{t+1} = J_{t+1}\psi_t/g_t$. Then $|J_{t+n}\cdots J_{t+1}\psi_t| = \prod_{j<n} g_{t+j}$, because ψ is propagated forward (`norm_wordMap_eq_prod_gain`); this is notation, not a constraint. The loss reads one secant along ψ at the last event of each training history, so it does not bound every g_t, and an operator-norm bound on a window is a different quantity. The trained F trackers have a median one-event top singular value of 1.73 and contract along runs (top factor 0.60). An earlier registered arm penalised one event along the top singular vector instead of ψ: it was exact on 8 of 10 seeds, with top factor 0.92 and 915 failing repeated-word pairs, 8.4 times M's 109 in the same run.

On S_5 the window acted as a gain. This was a registered question, and the answer is empirical, not a theorem. In the gain table of Section 8, one event at $G = 4$ is exact on 10 of 10 seeds; a window of 4 events with the weight scaled down by the same factor on 7 of 10; the window at full weight on 10 of 10; and one event at $G = 1$ on 8 of 10 (top factor 0.92, 797 failing repeated-word pairs against the window's 155).

Dose (exploratory). At $\sigma^2 = 0.1$ the delivered effective penalty strength, the push, is $c = 0.631\, c_0 + 0.013$ with $c_0 = \mu G \sigma^2/(1 + \sigma^2)$, the efficiencies measured on the basin. The dose sweep is Table 4; it was registered, but the push and the critical values are a post hoc reading of it, and the interval for c^* rests on basin counts of a few seeds. The basin still holds 1 seed at push 0.128 and none at 0.242, so c^* lies in $(0.128, 0.242]$. Exactness falls from 9 of 10 at push 0.472 to 5 of 10 at 0.931, where basin seeds return, so c^{**} lies between those pushes. $G = 4$ delivers 0.242. At a matched push, $\sigma^2 = 0.025$ is exact on 10 of 10 seeds, with 58 failing pairs against 94 at $\sigma^2 = 0.1$. A rule that sets σ to the measured cell radius is untested on a fresh setting. The one-event loss at $G = 4$ and $\sigma^2 = 0.1$ appears in three registrations, each with its own runs: the main text's exact-at-length table (top factor 0.59), the gain table (0.60, 96 failing repeated-word pairs) and the dose sweep (0.637, 94 pairs). Their differences are between cohorts.

1.6 Snap, and the certificate

The enumeration check applies to a tracker whose closure is a coordinatewise clip, or to a snapped tracker; it verifies the extracted automaton, not the continuous tracker. In this section a tracker is exact when its readout is right after every nonempty input.

Observation S5 (Enumeration) *Let P be the set of pairs (state, automaton state) reachable from the start, and P^+ the pairs after a nonempty word. The tracker is exact if and only if the readout is right on P^+. If P is finite, breadth-first search computes it with $|P| \cdot |X|$ evaluations of one event.*

Lean: `trackerExact_iff`, `reachedPairsPos_eq`, `reachedPairs_eq`, `reachedPairs_subset`; the operation count is proved on paper.

Proof Every pair in P^+ is the pair after some nonempty word, so exactness implies the condition. Conversely, the pair after any nonempty word lies in P^+, because P is closed under one event and P^+ is its one-event image. The search adds each pair once and applies each event once. □

When P is finite, the enumeration reads the transition and the readout at finitely many points, chosen adaptively, and it certifies exact trackers: the parity tracker $z \mapsto -z$ with the sign readout reaches two pairs and passes (`parity_enumeration`, `reachedPairs_congr`). So the conclusion of Corollary S2 cannot be widened to exactness in general; it holds for (C_r), and for exactness of a tracker that reaches a view the objective does not read.

Theorem S6 (A certificate must read the readout on the reached states) *Let V return "certified" or not after reading only values, and finitely many derivatives, of the transition, the closure and the readout at finitely many points. Suppose there are at least two outputs and V is sound for a class of trackers closed under changing the readout on a neighbourhood of one point. If V certifies a tracker, then every state that tracker reaches after a nonempty input is a point at which V read the readout.*

Lean: `certificate_reads_readout`, with adaptive reads.

Proof If some state h reached after a nonempty input, with automaton state q, has a neighbourhood that contains no point at which V read the readout, change the readout there so that h reads an output other than $o(q)$. The transition and the closure are unchanged, so h is still reached with q, and the new tracker is not exact. Every value V reads agrees, adaptive queries included, so V certifies a tracker that is not exact. □

The start state is excluded because exactness does not ask about the empty input: the check of Observation S5 reads the readout only on P^+, and on a tracker that never returns to its start state it never reads the readout there. Theorem S6 does not cover procedures that read the parameters, such as interval bounds; on the trained trackers those bounds were vacuous or divergent. It also binds only certificates that must be sound for a class closed under local changes of the readout, such as the class of all readouts. Such a certificate costs at least one readout evaluation per state reached after a nonempty input, and the search of Observation S5 uses $|P| \cdot |X|$ evaluations of one event. The trained trackers' heads are affine on views, and that class is not closed in this sense, since changing an affine map near one point changes it everywhere. For that class the bound does not apply: views span a subspace of dimension $d-1$, so the logits at d affinely independent views fix the head on views, and the readout at every enumerated state can then be computed rather than read (proved on paper).

Theorem S7 (Finite under perturbation forces a snap) *Let $\eta > 0$, let states update as $h_t = c(G(h_{t-1}, x_t) + e_t)$ with $\|e_t\|_\infty < \eta$, and let R_η be the set reachable under all such perturbations. If c is continuous and R_η is finite, then c is constant on the open ball of radius η about $G(h, x)$ for every reached h and every event x.*

Lean: `finite_perturbed_forces_snap`.

Proof The image of a connected ball under a continuous c lies in the finite set R_η, so it is constant. □

Theorem S8 (The clip) *Let c clip each coordinate to $[-1, 1]$, let $m(h, x) = \min_i |G_i(h, x)| - 1$, and let $\eta > 0$ bound the perturbations as in Theorem S7. If $m(h, x) \geq \eta$ on every edge out of the unperturbed reachable set R_0, then the perturbed reachable set equals R_0 and lies in the start state together with the corners $\{\pm 1\}^d$. If some edge has margin below η, the perturbed set contains a continuum. Under the first alternative, exactness under every such perturbation is equivalent to a correct readout on P^+.*

Lean: `clip_perturbed_eq`, `clip_perturbed_continuum`, `clip_robust_exact_iff`.

Proof If $|G_i| \geq 1 + \eta$ and $|e_i| < \eta$, then $G_i + e_i$ has the sign of G_i and modulus above one, so the clip returns the same corner. By induction every perturbed trajectory equals the unperturbed one. If some coordinate has margin below η, the clipped values fill an interval. The readout criterion is Observation S5. □

Corollary S9 (The certificate) *On a clip-closed tracker with finitely many events, if every reachable edge has positive margin and the readout is right on P^+, then the tracker is exact on every input at every length, and it stays exact under every per-step perturbation of the pre-closure vector smaller than the least margin over the reachable edges, which is positive.*

Lean: `clip_certificate`.

Proof The unperturbed reachable set lies in the start state together with the finitely many corners, and the events are finitely many, so there are finitely many reachable edges and their least margin m^* is positive. Theorem S8 with $\eta = m^*$ makes every perturbed run equal to the unperturbed one, and Observation S5 gives exactness. □

Proposition S10 (Contract, then snap, gives a radius) *Let $\Lambda > 0$, let $z = G(h, x)$ have margin at least $m > 0$, and suppose $\|G(h', x) - G(h, x)\|_\infty \leq \Lambda \|h' - h\|_2$ for all h' in the closed ball of radius ρ about h. Then the clip returns the same value at every h' with $\|h' - h\|_2 \leq \min(\rho, m/\Lambda)$, so the radius of carried-state perturbations that it absorbs is at least $\min(\rho, m/\Lambda)$. If every reachable edge, the edges out of the start state included, satisfies this with the same m^*, ρ and Λ, perturbations of the carried state by at most $\min(\rho, m^*/\Lambda)$ before each event leave the trajectory unchanged.*

Lean: `contract_snap_radius`, `contract_snap_trajectory`, for any metric on carried states.

Proof Inside that ball every coordinate of $G(h', x)$ keeps the sign of z and has modulus at least one, so the clip returns the same corner. The trajectory claim follows by induction over events. □

The search reads the margin, because the margin is a value at a reached state. It does not read Λ. Two balls on one contract-then-snap checkpoint were proved directly, by running every rendering of one generator through an ℓ_∞ ball of radius 0.025. Generator 8, the tight one, has least slack 0.031 over 16,384 renderings. Generator 2 has least slack 0.829 over 16,384 renderings. Both proof records pass. These are two edges of one checkpoint, not a certificate of the tracker. The batch extension of that argument has not landed, so a continuous certificate remains open.

Counted snap-in-the-loop training certified 0 of 20 trackers. Contract-then-snap certified 0 of 10 and was exact at length 1,024 on 8 of them. The one-event loss without the snap, in that same registration, was exact on 9 of 10. A longer contract-then-snap run certified 0 of 10 and was exact on 10 of them. The records say where the searches stopped. On the exact contract-then-snap seeds the snapped tracker reached all 120 states with 0 wrong readouts, but 351 to 4,532 reachable edges failed the margin test;

in the longer run at most 5 edges failed it, and all 10 searches stopped at their budgets. No further study of these failures was run. One comparison written in the lab notes was not pre-registered. It is not a result of this paper.

2 Pre-registration ledger

Predictions were committed before the counted runs. The table records the registration, what was predicted, and what happened, including every miss. Outcome numbers are macros generated from the records. On S_5 from scratch, M was registered after the isotropic perturbation, the label-free perturbation, shared-suffix partners and their mixture had been registered and run; their rows come first.

Table 1: Registrations, the prediction each one made, and the outcome, including the misses. Registration is the commit that fixed the prediction before the run; outcomes are exact seeds over seeds run unless stated. Rows from different registrations are different runs, so one configuration can carry different counts in different rows.

Study	Registration	Prediction	Outcome
Isotropic perturbation, S_5	`8577b41`	Exact on nearly every seed, and on at least as many as the earlier recipe	Missed: 6/10 at weight 1; the label-free variant 2/10. Rerun at equal dose in the last rows
Shared-suffix partners, S_5	`439974b`, `2a2fe1b`	More seeds exact than with plain partners	16/20, against the earlier recipe's 15/20; the edge over plain partners was not established
Mixture of partners, S_5 and Lean	`ed24635`	Exact on nearly every S_5 seed	Missed: 12/20
S_5 and Lean, windowed form M	`780eabd`	Aimed contraction exact where the earlier recipe misses; on Lean, a small per-event miss	S_5 20/20, earlier recipe 15/20. On Lean the miss bar of 0.02 was missed on two seeds
One event along the top singular vector	`056def3`	Matches the window on S_5	Missed: 8/10 exact, 915 failing repeated-word pairs, 8.4 times M's; the repeated identity fails on 10 seeds
One event along ψ, weight one	`e76f8c8`	Trains the contraction rate	Missed: 8/10 exact, top factor 0.92, 797 failing pairs against the window's 155. The counterexample loop on the window cut them to 59
Lean certificate	`64a2008`	Snapped trackers exact on every input; spectral radius below one	10/10 snapped; below one from depth three
One event against a window	`2152eb4`	The window is a gain, not a horizon	One event 10/10; window 7/10. In its repeated-word sweep the one-event form fails at least one word on 8/10 seeds, 96/1,300 pairs
Counterexample loop on the one-event form	`1400cd7`	The loop reduces repeated-word failures	Missed: 105 failing pairs against 96 without it
Dose sweep	`de51e4c`	The basin empties above a critical push and does not return	Basin count right at most doses. It missed the two lowest gains, and exactness collapsed at the highest. c^* and c^{**} are read post hoc

Study	Registration	Prediction	Outcome
Programs, addition, counting, reversible programs	`d1fdf4c`	One-event form matches the windowed form	It does on reversible programs, addition and counting. On programs it is 7/10 against 9/10
Matched smaller perturbation, programs	`e6feaa0`	A matched push at $\sigma^2 = 0.025$ matches the windowed form	Exact 9/10, never-wrong 8 against 7
Stories, first one-event retrain	`e98f203`	One-event form exact at the end	5/10. The sampler bug below voided the first launch
Stories, cell radius	`c3b9fa5`	Matching the perturbation to the cell matches the windowed form	Missed: 9/10 against 10/10, retuned on the same test sets
In-model supervised step	`6d9511e`	Free run exact on cards and reversible programs	512/512 at every length, 10 seeds. The prediction that it would miss at 4,096 failed in the other direction
In-model one-event loss	`6d9511e`	Adding the loss keeps the free run	Missed: the rule stopped 9 of 10. Perturbation measurement `f756d2e`: no new arm
Snap in the loop	`35041e7`	A clip certifies	0/20 certified
Contract, then snap	`46549c2`	At least one certified; the one-event control exact	Certified 0/10; exact 8/10. Control 9/10. One registered geometric prediction missed
Longer contract-then-snap, and the ball batch	`2e8d867`	Registered before the runs	Certified 0 of 10, exact 10. The ball cover of the earlier contract-then-snap checkpoint does not close the reachable set, so it certifies nothing
Noise	`cae5eec`	M tolerates the most noise on S_5; the theorem's bound holds on Lean and, in trajectory form, on every clean-exact checkpoint; the theorem orders the Lean arms; ordinary training with the fixed norm tolerates none; a single burst is absorbed more easily, by a ratio in [1.2, 5]; the bound's ingredients rank S_5 checkpoints	Held: M's median ε_{99} is 0.143 against 0.080 over the earlier recipe's exact seeds (0.071 with zeros imputed), and M's worst seed is at or above every other arm's median; the Lean bound holds on 13/13; the ranking gives $\rho = 0.87$ on 65, a heuristic use of the bound, which covers perturbations at block boundaries only. Missed: the trajectory form fails on O+Nc seed 2 (61/62); the theorem orders the Lean arms the wrong way round; O+N seed 2 reaches $\varepsilon_{99} = 0.102$, inside M's range 0.100–0.179; the burst ratio is 6.9, outside its range, though bursts are absorbed more easily on 111/111. On S_5 the bound itself is vacuous

Study	Registration	Prediction	Outcome
Balls	`1755ebe`	Radii, per-event miss, one-event inclusion, block ratio, final states in their balls, and perturbation decay on a trained S_5 tracker; repeated single moves break most seeds	Missed: median radius 0.187, below [0.25, 0.75]; miss 0.39 and 0.85 against 0.35 and 0.8; one-event inclusion 0.799 against 0.95; block images up to 1.40 radii against 0.6. Held: every final state in its ball; perturbation at a median 0.072 after 8 events (upper quartile 0.127); repeated single moves break M on 13/20 seeds and the earlier recipe on 19/20
Cube	`a301269`	A factorised tracker exact far past training	10/10 at 4,096 and at 10,000
Frontier models	`c1486af`	Exact where a short suffix fixes the answer, and not otherwise	Held on programs at 1,024. Missed on S_5, on counting at 4,096, and on addition at 4,096
Five tasks, separate models	`74de3ea`, `8a590d5`	Never-wrong at every length, three seeds	Held on the 21 task-seeds of the first wave, on the cube and on Lean. Failed on eight-variable programs, 0/3
Joint model	`ffdf6f7`	At least one of two seeds through the joint gate	Gate 0 and 0, 2 seeds each. Long tests were not run
Isotropic perturbation at equal dose, S_5	`dd8199f`	With M's window, size and weight, and in F's form, exact on fewer seeds than the aimed arm, with a larger top factor and more failing repeated words	Held: 14/20 against M's 20/20, top factor 0.88 against 0.70, failing pairs 1,683/2,600; F's form 3/10 against F's 10/10, failing pairs 1,164/1,300
Closure alone and ordinary training, Lean, new seeds	`dd8199f`	Ordinary training exact on almost no new seed, the closure on nearly all, at both test lengths	Held: with the earlier seeds, O 0/10; O+Nc 10/10 at 1,024 and 9/10 at 4,096
Stories retune, fresh test set	`d72ed85`	Most seeds exact on stories never scored before, with exactness, per-step error and never-wrong fraction close to the first test set	Held: 8/10 exact, against 9/10 on the first set
Closure alone and ordinary training, reversible programs, addition and counting, new seeds	`1acb2d0`	Ordinary training exact on almost no new seed; the closure on nearly all in reversible programs and in addition on both test sets, and on some in counting; a median top factor near one without the closure and below it with	Held: with the earlier seeds, O 0/10 in reversible programs, 0/10 in addition (0/10 on long carries) and 0/10 in counting; O+Nc 9/10, 10/10 (10/10) and 4/10; median top factors 1.005–1.009 without the closure and 0.566–0.936 with it, where recorded
Programs retune, retrained, fresh test set	`b020bf6`	Retrained with the weights saved, most seeds exact on the original test set and on a fresh set sharing no program, the two counts within two seeds	Held: 9/10 on the original set and 9/10 on the fresh set, against 9/10 in the first run; median top factor 0.051 against 0.052

The stories sampler bug is part of the ledger. The first one-event launch drew its validation and test stories with a generator that reset inside the comprehension, so each checkpoint was chosen on 512 copies

of one story. Those runs are not reported. The retrain on distinct stories is the 5/10 line. The cell-radius retrain is the line below it.

3 Repeated words

Held-out problems are drawn from the training distribution, where a long word almost never repeats one short pattern. Repeated words are the adversarial case, and the registered sweeps measured them on S_5.

Protocol. The main sweep runs 130 words, each a repetition of one pattern (the identity, transpositions, and words of period up to four), for 1,024 events from 8 random prefixes in 8 renderings each. A word fails on a seed when fewer than a fraction 0.99 of its runs are right after every event. The held-out sweep uses 64 words of longer periods. The ball study's sweep repeats each of 11 single moves, the identity among them, for 1,024 events.

Table 2 Repeated words on S_5. Failing pairs are (seed, word) pairs that fail; identity counts seeds that fail the repeated identity; held out counts failing pairs on the held-out sweep. Rows with 130 words come from the registered one-event studies and the equal-dose isotropic control (last two rows); rows with 11 words from the ball study. – marks a sweep that was not run.

Objective	Seeds	Words	Failing seeds	Failing pairs	Identity	Held out
F: one event, secant gain four	10	130	8	96	3	0
F with the counterexample loop	10	130	9	105	2	2
one event, secant gain one	10	130	10	797	8	305
window of four events, one term	10	130	10	155	8	9
window of four events with the loop	10	130	6	59	2	2
M	10	130	10	103	8	0
one event along the top singular vector	10	130	10	915	10	–
M, single generators	20	11	13	22	6	–
earlier recipe, single generators	20	11	19	49	9	–
isotropic, M's dose	20	130	20	1,683	19	–
isotropic, F's form	10	130	10	1,164	10	–

What breaks. Every objective in the table fails some repeated word on most seeds, including F and M, which are never wrong on held-out problems from the training distribution. On a random word the per-event misses point in varying directions and are pulled back; on a repeated word the same maps apply in the same order, so the miss can add up along the cycle. On the measured M seed the failing single moves settle 2.7–3.1 radii from their class centre, outside the ball, although contraction along each such word was measured below one. First wrong events come late: for M at event 16 at the earliest and 38 at the median, for the earlier recipe at 29 and 122.

The counterexample loop. Adding failing repeated words to training cut the window's failing pairs from 155 to 59, but not F's (96 without the loop, 105 with it). On held-out problems the window acted as a gain; on repeated words, with the loop, it may do more. We do not have an objective that is right on every repeated word.

Aimed against isotropic. An isotropic perturbation with M's window, size and weight, and one in F's form with the kick along a random tangent direction instead of the power-iterated one, were registered and run on S_5 from scratch at equal dose. Both fail far more repeated words than the aimed arms in the rows above (1,683 of 2,600 and 1,164 of 1,300 pairs), and the first contracts less (median top factor 0.88 against M's 0.70).

4 Stories and programs

Stories. Exact means every object in its right container after the last sentence. Because an object's final place is set by its last move, exactness at the end measures less than tracking, and the never-wrong fraction is the stricter score. M is exact at the end on 10 of 10 seeds, never-wrong fraction 0.58 (range 0.35–0.79), per-step error 0.0034. Certified equivalence with the closure is exact on 10 of 10, never-wrong fraction 0.65. The earlier recipe is exact on 10 of 10. Closure alone is exact on 0 of 3 and ordinary training on 0 of 3, so on stories certified equivalence is needed. F at its default settings, as in the main text's table, is exact at

the end on 5 of 10 seeds, never-wrong fraction 0.20. Retuned so that σ matches the measured cell radius (0.117, gain 26.9), it is exact on 9 of 10, never-wrong fraction 0.474, per-step error 0.0053. The retune was chosen after the test sets had been scored. On a fresh test set, drawn and registered before it was scored and sharing no story with the first, the same checkpoints are exact on 8 of 10 seeds.

Programs. Many final states are fixed by a short suffix, so the setting tests less than its length suggests. M is exact on 9 of 10 seeds and the earlier recipe on 8 of 10; without the closure (NS+N) 2 of 10, closure alone 0 of 3, ordinary training 0 of 3. F at its default settings is exact on 7 of 10 seeds (interval 0.35–0.93). At $\sigma^2 = 0.025$ and gain 11.5 it is exact on 9 of 10, the same as M, and never-wrong on 8 of them against M's 7; the top factor is 0.052 against 0.420. That setting too was chosen after the test sets had been scored. Its checkpoints were not saved, so the same arm was retrained on the same seeds with the weights saved. The retrain is exact on 9 of 10 seeds on the original test set and, on a fresh test set drawn and registered before it was scored and sharing no program with the first, on 9 of 10; the same seed misses in the first run and on both sets.

5 Multiplicity

Each drop-one comparison is a one-sided Fisher exact test of M against M without one part. The family is the 22 tests below, including M against the earlier recipe on S_5; the adjusted values are Holm's step-down correction at 0.05. 7 survive. M against certified equivalence with the closure on S_5, at 20 of 20 against 12 of 20, has raw $p = 0.002$; it was not in the registered family and is reported uncorrected. The equal-dose isotropic controls were registered as count predictions, not as tests. Against the aimed arm, the one at M's dose (14 of 20 against 20 of 20) has raw $p = 0.010$ and the one in F's form (3 of 10 against 10 of 10) raw $p = 0.002$, outside the family and uncorrected.

Table 3 Drop-one comparisons with raw and Holm-adjusted one-sided p. Counts are exact seeds.

Setting	Comparison	Without	M	Raw p	Holm p	Survives
Reversible programs	without aimed contraction	10/10	10/10	1.000	1.000	no
Reversible programs	without certified equivalence	7/10	10/10	0.105	1.000	no
Reversible programs	without closure	0/10	10/10	5.4×10^{-6}	1.2×10^{-4}	yes
S_5	without aimed contraction	5/10	10/10	0.016	0.211	no
S_5	without certified equivalence	4/10	10/10	0.005	0.081	no
S_5	without closure	9/10	10/10	0.500	1.000	no
Programs	without aimed contraction	9/10	9/10	0.763	1.000	no
Programs	without certified equivalence	3/10	9/10	0.010	0.138	no
Programs	without closure	0/10	9/10	6.0×10^{-5}	0.001	yes
Lean	without aimed contraction	10/10	10/10	1.000	1.000	no
Lean	without certified equivalence	10/10	10/10	1.000	1.000	no
Lean	without closure	10/10	10/10	1.000	1.000	no
Addition	without aimed contraction	10/10	10/10	1.000	1.000	no
Addition	without certified equivalence	8/10	10/10	0.237	1.000	no
Addition	without closure	0/10	10/10	5.4×10^{-6}	1.2×10^{-4}	yes
Counting	without aimed contraction	10/10	10/10	1.000	1.000	no
Counting	without certified equivalence	1/10	10/10	6.0×10^{-5}	0.001	yes
Counting	without closure	0/10	10/10	5.4×10^{-6}	1.2×10^{-4}	yes
Stories	without aimed contraction	10/10	10/10	1.000	1.000	no
Stories	without certified equivalence	0/10	10/10	5.4×10^{-6}	1.2×10^{-4}	yes
Stories	without closure	0/10	10/10	5.4×10^{-6}	1.2×10^{-4}	yes
S_5	earlier recipe in place of M	15/20	20/20	0.024	0.283	no

6 Architecture baselines on Lean

These are existing registered records, not new runs. A depth-eight transformer and a DeltaProduct network (the Householder-product linear recurrence of Siems and colleagues), and DeltaProduct with the earlier recipe's terms, were tuned on validation at the training length, 3 seeds each. All three fit the training length: the worst seed is exact on a fraction 0.977, 0.986 and 0.994 of chains of 8 rewrites. At 1,024 rewrites the best seed is exact on 0.006, 0.010 and 0.045. Baselines on S_5 with matched tuning, such as a Mamba or a negative-eigenvalue linear recurrence, were not run.

7 Protocols

Trackers have width 128. A run is exact at a length when the final state is right on at least 0.99 of 512 held-out problems. A problem is never wrong when the state is right after every event. Checkpoints are selected by validation at the training length, after every term is on, never at test lengths; S_5 from scratch reports the final weights. Counted runs were registered before launch. Hosts differ; records store AVX-512 and AVX2 flags and a hash of the flag string, and we claim no cross-host bit identity.

Units. Views, radii, σ and ε are in the units of Section 1: chords between views on the centred unit sphere, while the code carries the state at centred norm $\sqrt{d}$. F uses $\sigma^2 = 0.1$, so $\sigma = 0.32$; M uses $\sigma = 0.1$.

Trackers. The state w starts from a learned vector. On S_5 from scratch the tokens of an event are embedded with a position code and layer-normed into x, and

$$w' = w + \mathrm{XAttn}(\mathrm{LN}\, w,\ x), \qquad w \leftarrow \mathcal{N}\big(w' + \mathrm{MLP}(\mathrm{LN}\, w')\big),$$

where XAttn is multi-head cross-attention from the state to the event's tokens, so the tokens of one event are pooled by attention. Lean applies the same block four times per event. Through the frozen model, x is the event text's layer-12 hidden states; the state cross-attends to them, and the attended vector and the state pass through a deeper MLP that is added back residually. The readout is a layer norm and a linear head on the closed state. The centred closure is $\mathcal{N}(w) = \sqrt{d}\,(w - \bar{w}\mathbf{1})/|w - \bar{w}\mathbf{1}|$, with $\bar{w}$ the mean coordinate. Lean uses the RMS closure $\sqrt{d}\,w/|w|$: its configuration was registered before the centred closure was introduced and was not rerun, so on Lean the state keeps a mean that the readout's layer norm removes and the transition does not.

Losses. The readout cross-entropy is averaged over every event of a history. F is read at the last event: the base state, both perturbed images and the partner's final state carry gradients, and ψ does not. It starts from a Gaussian vector projected onto the tangent space at the first state and is carried along the history by the closed tangent map under stop-gradient. AIM perturbs the state after a burn-in of 8 events of power iteration and runs it through the next 4 events with live parameters; the base state, the direction and the clean target are under stop-gradient. Histories too short for a term carry no weight in it. Certified equivalence is the centred cosine distance between the final views of a history and its partner, with gradients through both. Next-state prediction forms $\hat{h}_{t+1} = h_t + \mathrm{MLP}(\mathrm{LN}[x_{t+1}, h_t])$, with h the readout's layer-normed state and an MLP with GELU activations and no biases; its loss is the smooth-L_1 distance to h_{t+1} under stop-gradient plus a KL divergence between the head's outputs at h_{t+1}, under stop-gradient, and at $\hat{h}_{t+1}$, through detached head weights.

Partners. Partners are built by construction and checked, not found by chance. On S_5 a partner is a uniformly random word with the same product, at a uniformly random feasible length. In reversible programs it is a random prefix, NOT gates that fix the difference and a random suffix, run by the interpreter. In addition and counting it is a random prefix of random length followed by the item that lands on the target state. On Lean it is a direct encoding of the expression a prefix reaches, followed by the chain's remaining rewrites, and the kernel checks it. In stories it is a prefix of random length whose last move is chosen to reach the target world. On the cube some partners insert words that return a cube to its state and others join a random prefix to a solver's path to the target; the simulator checks both.

Schedules and selection. S_5 from scratch has no delay, because in its registered pilot the state-level term did not slow the formation of class codes, and it reports the final weights because its registered protocol fixed them.

Reversible programs. Each statement is a NOT, CNOT or Toffoli on four bits. Training lengths 4 to 16, test lengths up to 4,096, 512 programs each. The frozen Qwen2.5-Coder-0.5B, layer 12, encodes each line. Closure is centred. A partner is an independent program the interpreter runs to the same final state. The state-level terms are delayed 800 steps and ramped over 400, in a run of 16,000 steps.

S_5, from scratch. Events are transpositions among distractor tokens, read as tokens. 218,488 parameters, 4,000 steps, training words of 16 events, test to 1,024. Partners are independent words with the same product. The windowed form uses a window of 4 events, perturbation $\sigma = 0.1$, weight 10, burn-in 8.

S_5, read as text. The same words, one move per line, through the frozen code model. Training length 16, test 4,096.

Lean. Each event is a rewrite tactic; the Lean 4 kernel certifies partners, a direct encoding of the expression followed by the remaining tactics. 227,773 parameters, 12,000 steps, training chains of 8 rewrites, test 1,024 and 4,096. The windowed form uses 3 events, burn-in 3, and the root-mean-square closure. State-level terms are off for the first 20 percent of training and ramp over the next 10 percent.

Addition and counting. Addition reads aligned digit pairs, least significant first; the state is the pair of the current sum digit and the carry, and the readout reads both. Counting reads a marked token and outputs the count modulo ten. Training 4 to 32 items, 3,000 steps, test 4,096. Partners come from Python integers. Delay 400, ramp 200. Long-carry additions, with the carry chain running the whole length, are reported with the addition row.

Stories. Each sentence moves one object among containers, in varied English, read by the frozen Qwen2.5-0.5B. A simulator certifies partners. Training 16 events, test 1,024. Exact means every object right after the last sentence. The cell-radius follow-up uses $\sigma = 0.117$ and gain 26.9.

Programs. Eight variables, read by the frozen code model. Many final states are fixed by a short suffix, so the setting tests less than its length suggests. The matched-perturbation follow-up uses $\sigma^2 = 0.025$ and gain 11.5.

Cube. Eighteen face turns, read by the frozen Qwen2.5-0.5B. One slot per movable piece, shared weights, 4,000 steps, batch 32, training scrambles of 32 moves. A simulator certifies partners. The reported bar is 4,096, and the runs were also scored at 10,000.

In-model step. The state is one slot per variable, each the frozen model's own layer-12 residual at the newline of a line that prints that variable's value; on cards the variables are the positions of the named cards, the start is each card in its own position, and a statement swaps two cards or does nothing. A step writes the closed slots into the residual stream at layer 12 in place of those lines, runs the remaining layers on the statement, applies a linear map with bias, initialised by least squares, to the outputs, and closes each slot. The supervised step (L-TF) is teacher-forced: its input is the true state's code and its target the true next state's code, by centred cosine, plus readout cross-entropy. It is then run free from the canonical start. The one-event loss was added on a ramp of 1,000 steps, and a seed was stopped if its free-running fraction fell by more than 0.02. Cards use five slots and reversible programs four. Eight-variable programs use the same step; their final-state accuracy is not the never-wrong count, because a later assignment overwrites a variable.

Separate models, five families. The same supervised step, one model per task, three seeds. The first registration had seven tasks, five arithmetic ones, cards and reversible programs, so 21 task-seeds; programs, the cube and Lean were the second. Test length 4,096, 512 problems per length. The cube is also scored at 10,000. The joint model finished 2 seeds on each arm. Its gate flags are 0 and 0, and a long test is scored only after the gate.

8 Extended tables

The main text's exact-at-length table is not repeated. The rows below are the follow-ups and the per-task in-model counts.

Table 4 Dose sweep on S_5 at $\sigma^2 = 0.1$. Push $c = 0.631\, c_0 + 0.013$. Basin seeds are those that have not left the marginal basin. Failing pairs are out of the registered word list.

Gain	Push	Basin seeds	Exact	λ	Failing pairs
0	0.013	4	7/10	0.920	909
0.25	0.027	4	6/10	0.886	823
0.5	0.042	4	7/10	0.900	816
1	0.070	4	9/10	0.872	742
1.41	0.094	3	9/10	0.825	683
2	0.128	1	10/10	0.842	560
4	0.242	0	10/10	0.637	94
8	0.472	0	9/10	0.415	134
16	0.931	2	5/10	0.544	587

Table 5 Separate supervised steps, three seeds. The middle column is seeds never-wrong at every tested length. The right column is the never-wrong count at 4,096, or at 10,000 for the cube.

Task	Seeds	Never-wrong count
Addition	3/3	512/512
Subtraction	3/3	512/512
Multiplication	3/3	512/512
Remainder	3/3	512/512
Running total	3/3	512/512
Cards	3/3	512/512
Reversible programs	3/3	512/512
Eight-variable programs	0/3	103–499/512
Rubik's cube	3/3	512/512
Lean rewrites	3/3	512/512

The supervised in-model step is never-wrong on 512 of 512 at 4,096, on all 10 seeds, on both tasks. This is a linear map trained teacher-forced on the model's codes, with no Nerode term; it does not show that the language model tracks state. On cards seed 0 the write snaps errors: a perturbed code lands as close to its target as a clean one (1.1×10^{-4} against 1.1×10^{-4}). The one-event loss stopped 9 of 10 seeds, and one that finished tied. Perturbations at $\sigma = 0.316$ still read as the same state on 1,280 of 1,280 reversible-program rows and on 1,228 of 1,280 card rows. The median cell radius along the forward direction is 0.40 on reversible programs and 0.18 on cards. Both sit at or above the rule's refute line, and no scaled-perturbation arm was registered.

9 Frontier protocol

Three models, Claude Fable 5.1, GPT-6 Astra and Gemini 4 Argon, answered with no tools. The standard budget is 64,000 output tokens including thinking. A second pass used their largest budgets, 128,000 tokens for Claude Fable 5.1 and GPT-6 Astra, and a long decode for Gemini 4 Argon. Each cell has between 1 and 5 problems per model (Table 6), drawn from the same generators as the trackers. A miss is a wrong final answer or a stop at the budget; the S_5 misses at 1,024 are wrong answers inside the budget, 4 of 15 exact at the standard budget and 3 of 9 at the largest. Reversible programs are exact on all 12 problems at 1,024, and at 4,096 on 2 of 6 at the standard budget and 7 of 9 at the largest. Addition at 4,096 is 4 of 9. The cube at 1,024 turns is 0 of 8, and GPT-6 Astra solves 3 of 3 at 256 turns. Our trackers generate no tokens. Over all cells but the cube, 7 of the 34 misses are budget stops; the rest are wrong answers given inside the budget.

Every prompt forbids tools and asks for the final answer inside answer tags. The S_5 prompt, verbatim, with n the length and the moves one per line:

> Five cards A, B, C, D and E lie in a row in positions 1 to 5, in that order (A in position 1, E in position 5). Below are *n* moves, applied in order. "swap i j" exchanges the cards in positions i and j; "no swap" leaves the row unchanged. Work it out yourself: do not use any tools or code. You may reason step by step. At the end, give the final order of the cards from position 1 to position 5 inside <answer></answer>, for example <answer>CAEBD</answer>.
>
> Moves: *moves*

The other prompts are in `research/frontier/frontier.py` at `1579e4c2` and, for the cube, in `research/cube_frontier.py` on the cube branch.

Table 6 Frontier cells. Problems per model and exact counts are given for Claude Fable 5.1, GPT-6 Astra and Gemini 4 Argon in that order; one number means the same count for all three. The last column counts misses that stopped at the budget, and for the cube problems left unanswered. – marks a model not run.

Setting	Length	Budget	Problems	Claude	GPT	Gemini	Stops
Reversible programs	64	standard	5	5	5	5	0
Reversible programs	256	standard	5	5	5	5	0
Reversible programs	1,024	standard	4	4	4	4	0
Reversible programs	4,096	standard	2	0	2	0	4
Reversible programs	4,096	largest	3	3	2	2	0
S_5 words	64	standard	5	5	4	5	0
S_5 words	256	standard	5	5	4	3	0
S_5 words	1,024	standard	5	1	0	3	0
S_5 words	1,024	largest	3	2	0	1	0
Lean rewrites	1,024	standard	3	3	3	3	0
Addition	4,096	standard	3	2	2	0	3
Counting	1,024	standard	5	4	5	5	0
Counting	4,096	standard	3	3	3	3	0
Programs	256	standard	5	4	5	5	0
Programs	1,024	standard	3	3	3	3	0
Rubik's cube	16	standard	3/3/3	2	3	3	0
Rubik's cube	64	standard	3/3/3	3	3	1	0
Rubik's cube	256	standard	1/3/2	0	3	0	3
Rubik's cube	1,024	standard	3/2/3	0	0	0	6

10 Compute and cost

Totals below are the ledgers as written, not a recomputation. The one-event study settled 4.15 dollars. The dose sweep settled 5.33. The drop-one and table runs on programs, addition, counting and reversible programs settled 65.36. Frontier calls cost 45.39. The first wave of separate in-model models settled 41.98, inside a larger cap that later waves also drew on. The stories cell-radius retrain settled 5.894, and its fresh test set 1.914. The equal-dose isotropic control and the Lean seeds for the closure alone settled 4.29. The seeds for the closure alone and ordinary training in reversible programs, addition and counting settled 8.02, and the programs retrain with its fresh test set 7.99. The snap and contract-then-snap runs settled 5.34 before the longer run. The ball batch has not landed. Noise measurements ran on CPU. Counted training ran on cloud containers, with or without a GPU, and the records name the host.

11 Use of large language models

Language models did much of this work under the author's direction. Anthropic's Claude models, used directly and through Cursor's agents, wrote most of the code, ran the registered experiments on cloud hardware, drafted the proofs in Section 1 from the working notes, and drafted the manuscript. The author chose the questions, approved every registered prediction and every claim, and is responsible for the content. Predictions were committed before each counted run. Every number in the main text and in this note is generated from run records by `numbers.py`, and `AUDIT.md` maps each claim to its record or proof. GPT-6 Astra and Gemini 4 Argon appear only as test subjects. Every figure is drawn by code from the records. No image was generated by a model. The core theorems are machine-checked in Lean 4 with Mathlib, in a formalisation written with the same models: Theorem S1 and its likelihood consequence, the readout bounds and limit, Theorem S3 with the counterexample without non-expansion, Corollary S2, Theorem S4 and the failure of its converse on a line, Lemma S1 with the secant bound and the growth identity, Observation S5 and the parity enumeration, and Theorems S6, S7 and S8, Corollary S9 and Proposition S10, each as stated in Section 1, where the Lean theorem is named beside it. Proved on paper only: that F and reached-state objectives read in the sense of Corollary S2, and the adaptive form of its last statement; the drift statement after Theorem S3, and the failure of (ii) without non-expansion; the failure of the converse on the circle; injectivity in the radius-zero case; the operation count of Observation S5; and the cost of a certificate for affine heads. The author is responsible for every argument.